

Quarterly Iteration Cycle

GODMODE V2

V2-godmode upgrade with ML scoring retraining (XGBoost), Bayesian A/B framework (PyMC3), multi-armed bandit (Thompson sampling), causal impact analysis, dbt warehouse schema, reverse ETL, QBR template, swarm governance RACI.

DOCUMENT TYPE	Operational Runbook — GODMODE V2
AUDIENCE	Marketing-Ops (1 FTE) + Data/ML Engineering + LO Ops + Compliance
CLASSIFICATION	Internal · Ongoing · Quarterly Cadence
CADENCE	Quarterly (Q2: April, Q3: July, Q4: October, Q1: January)
GODMODE UPGRADE	ML retraining (XGBoost + MLflow) + Bayesian A/B (PyMC3) + multi-armed bandit + causal impact + dbt warehouse + reverse ETL
PRIMARY OUTPUT	Updated persona library + recalibrated scoring weights + refreshed case files + ML model registry
CRITICAL RULE	Never deploy unvalidated scoring changes mid-quarter — always at quarter boundary

Quarterly Iteration Cycle Runbook — GODMODE (V2)

Document ID: D4-GODMODE **Predecessor:** 04_Quarterly_Iteration_Runbook.pdf (V1 — 6-agent re-execution + recalibration protocol) **Upgrade thesis:** V1 operated on rule-of-thumb recalibration and 6-agent re-execution. GODMODE replaces heuristic recalibration with **production ML retraining, Bayesian A/B testing, multi-armed bandit creative rotation, causal impact measurement on geo-holdouts, statistical process control on persona FDI, LTV/CAC modeling per persona, a dbt-modeled star-schema warehouse, reverse ETL to ad platforms, and a governance RACI** that codifies who owns each operational lever. V1 was an operational checklist. GODMODE is a data-science operating system.

Owner: Marketing-ops leadership (RACI Part 12) **Cadence:** Quarterly iteration cycle (10 business days) + annual swarm refresh (Q2) **Audience:** Marketing-ops · LO operations · Compliance/Reg B counsel · Data/ML engineering · Executive leadership **Compliance anchors:** Reg B § 1002.5(b)(1) (no "rather-not-say" penalty) · Reg B § 1002.4 (no protected-class proxies) · Reg B § 1002.9 (adverse-action notice on hard-exit) · ECOA · GLBA · Meta Special Ad Category (Housing) · Google Ads housing/credit policies · state UDAAP

Table of Contents

- Part 1 — Iteration Cycle Architecture (refresh from V1)
- Part 2 — ML Scoring Model Retraining (THE WOW ELEMENT)
- Part 3 — Bayesian A/B Testing Framework (PyMC3)
- Part 4 — Multi-Armed Bandit Creative Rotation (Thompson Sampling)
- Part 5 — Causal Impact Analysis (Geo-Holdout)
- Part 6 — Cohort Retention Curves (Funded-Loan Repeat Borrowing)
- Part 7 — Persona FDI Drift Detection (Statistical Process Control)
- Part 8 — LTV/CAC Modeling per Persona
- Part 9 — Data Warehouse Schema (dbt star schema)
- Part 10 — Reverse ETL Pipeline (Census / Hightouch spec)
- Part 11 — Lookalike Audience Refresh Cadence
- Part 12 — QBR Deck Template + Swarm Governance

Part 1 — Iteration Cycle Architecture (Refresh from V1)

1.1 Cadence

Cycle	Window	Business days	Trigger
Quarterly iteration cycle (QIC)	First 2 calendar weeks of each fiscal quarter (Q1 Jan 1–14, Q2 Apr 1–14, Q3 Jul 1–14, Q4 Oct 1–14)	10 business days	First business day of quarter
Mid-quarter stability window	Weeks 3–13 of each quarter	~9 weeks	Hard rule — see § 1.5
Annual swarm refresh (ASR)	First 4 calendar weeks of Q2 (Apr 1–28)	20 business days (overlaps Q2 QIC)	First business day of Q2

Hard rule: No scoring-weight changes, no persona-library reordering, no creative-library deprecation, no geo-tier re-tiering during the mid-quarter stability window. The only exception is documented under § 1.6 (Exception Process for Mid-Quarter Changes).

1.2 Six-Agent Re-Execution Sequence (every QIC)

The QIC re-executes 6 of the 10 swarm agents against the most recent quarter of funded-loan + declined-loan data. The 6 agents are CF-01, GL-02, AP-03, NP-04, SA-05, TS-10 — the core production agents. EG-06, GS-07, FF-08, AC-09 are refreshed annually (see § 1.3).

Day	Phase	Agents	Output	Owner (RACI Part 12)
1	Data harvest	CF-01 (case-file harvester) + Data/ML (warehouse extract)	20+ new case files added to <code>dim_cases</code> table + raw <code>stg_leads</code> extract	Data/ML R/A, Marketing Ops C
2	Guideline refresh	GL-02 (guideline normalizer)	Updated <code>dim_lenders</code> rows (withdrawals, program changes, FICO/LTV/DSCR band updates)	LO Ops R/A, Compliance C, Data/ML C
3	Pattern re-mine	AP-03 (approval-pattern miner) + NP-04 (decline-pattern miner)	Updated <code>fact_approval_clusters</code> + <code>fact_decline_clusters</code> rows; FDI overlay re-scored	Data/ML R/A, Marketing Ops C

Day	Phase	Agents	Output	Owner (RACI Part 12)
4	Persona library sync	SA-05 (sponsor archetype synthesizer)	12 personas (SA-001 → SA-012) re-scored against new evidence; new personas proposed if drift $>3\sigma$ (Part 7)	Marketing Ops R/A, Compliance C
5	Scoring retrain	TS-10 (targeting & scoring generator) + ML retrainer (Part 2)	New TS-10 model version (XGBoost + logistic baseline) registered to MLflow; shadow-mode deploy	Data/ML R/A, Marketing Ops C, Compliance I
6	Bayesian A/B test design	Part 3 framework	New A/B test plan for Q+1 (creative, persona, channel, landing-page)	Marketing Ops R/A, Data/ML C
7	Multi-armed bandit refresh	Part 4 framework	Thompson sampler posteriors reset to uniform Beta(1,1) on any newly added creative; existing posteriors carried forward	Marketing Ops R/A, Data/ML C
8	Causal-impact / geo-holdout review	Part 5 framework	Last quarter's geo-holdout lift report; new holdout design for Q+1	Data/ML R/A, Marketing Ops C
9	Cohort retention + LTV/CAC refresh	Parts 6 + 8	Updated retention curves per persona; updated LTV/CAC ratios per persona	Data/ML R/A, Marketing Ops C, LO Ops C
10	QBR deck finalization + sign-off	Part 12	QBR deck (15 slides) + RACI sign-off block; deployment checklist for Q+1	Marketing Ops R/A, Leadership A, Compliance C, Data/ML C, LO Ops C

1.3 Annual Swarm Refresh (Q2 only — adds EG-06, GS-07, FF-08, AC-09)

Day	Phase	Agents	Output
1–10	QIC for Q2 (core 6 agents)	per § 1.2	per § 1.2
11–13	Edge-case refresh	EG-06 (edge-case gold miner)	EG-001 → EG-008 edge-case personas re-evaluated; new edge cases proposed if specialty-lender programs changed

Day	Phase	Agents	Output
14–16	Geo-segment refresh	GS-07 (geo-segment correlator)	GS-07 geo tiers re-tiered; STR-market permissiveness re-verified; insurance-crisis overlays refreshed
17–18	Funnel friction refresh	FF-08 (funnel friction mapper)	12-question intake form re-validated; 16 hard-exit rules reviewed; new friction patterns added if Q-1 cohort shows new friction clusters
19–20	Creative library refresh	AC-09 (ad hook & copy reframer)	V2 creative library (120 hooks + 20 lead magnets + 100 objection destroyers + 60 repel elements + 20 landing pages + 10 YouTube scripts) refreshed; new hooks for any new personas; deprecated hooks retired

1.4 Critical Rule — No Mid-Quarter Scoring Changes

Scoring model version, persona-library FDI weights, and tier-routing bands are **frozen** at end of day 10 of each QIC. They remain frozen through the rest of the quarter. Rationale:

- 1. Statistical validity of A/B tests:** Mid-quarter scoring changes invalidate the steady-state assumption of Bayesian A/B tests (Part 3) and Thompson samplers (Part 4). Without a frozen baseline, "did the variant beat the control?" is unanswerable.
- 2. Causal-impact integrity:** CausalImpact (Part 5) assumes the only intervention during the post-period is the ad campaign. A scoring change in week 7 contaminates the lift estimate.
- 3. Cohort comparability:** Cohort retention curves (Part 6) compare apples-to-apples quarters. Mid-quarter scoring changes break cohort comparability.
- 4. Compliance traceability:** Reg B § 1002.9 adverse-action notices must reference the scoring model version in effect at the time of the adverse decision. Mid-quarter model changes create an audit-trail nightmare.

1.5 Exception Process for Mid-Quarter Changes

A mid-quarter scoring change is permitted **only** under one of the following triggers, and only with documented sign-off:

Trigger	Required sign-off	Max scope of change
Compliance mandate (Reg B / ECOA / state UDAAP directive)	Compliance R/A + Leadership A + outside ECOA counsel	Hex-rule additions only (no weight changes); model version bumped with audit-trail entry
Specialty-lender program withdrawal (a GL-02 lender exits DSCR)	LO Ops R/A + Compliance C	Routing-rule update for affected personas; tier bands unchanged
Data-warehouse schema break (column renamed / dropped by upstream)	Data/ML R/A + Marketing Ops I	Hotfix to warehouse loader; no scoring change
Model drift emergency (Part 7 SPC detects $>4\sigma$ drift on ≥ 2 personas)	Data/ML R/A + Leadership A + Compliance C	Model rollback to prior MLflow version; full audit-trail entry; expedited QIC scheduled

Every exception is logged to `fact_iteration_exceptions` with: timestamp, trigger, sign-offs (name + role + timestamp), scope of change, MLflow model version before/after, and a 200-word rationale. The exception log is reviewed at the next QIC and included in the QBR deck (Part 12, slide 12).

1.6 QIC Entry & Exit Criteria

Entry criteria (all must be true on day 1):

- Prior quarter's funded-loan data is loaded to `fact_leads` and reconciled to the LOS (loan origination system) within ± 2 loans.
- Prior quarter's ad-spend data is loaded to `fact_ad_spend` and reconciled to Meta Ads Manager + Google Ads + LinkedIn Campaign Manager + YouTube / Google Ads video within $\pm \$500$.
- MLflow tracking server is operational; prior-quarter model version is tagged `production`.
- dbt warehouse models all pass `dbt test` (no failed data-quality tests).
- All four RACI sign-off blocks from prior quarter's QBR are complete.

Exit criteria (all must be true on day 10):

- New TS-10 model version is registered to MLflow, $AUC \geq$ prior-quarter $AUC - 0.02$ (no regression rule), and shadow-mode deploy complete.
- QBR deck (15 slides) is reviewed and signed off by Marketing Ops, LO Ops, Compliance, Data/ML, Leadership.
- Q+1 A/B test calendar is finalized (≥ 6 tests planned, each with sample-size estimate from Part 3).
- All exception log entries from prior quarter are reviewed and either resolved or carried forward with owner + due-date.

1.7 Tooling Stack (GODMODE add to V1)

Layer	Tool	Purpose
Data warehouse	Snowflake (or BigQuery)	Star schema (Part 9), conformed dimensions
Transform	dbt (data build tool)	Version-controlled SQL models, tests, docs
ML platform	MLflow + Python 3.11 + scikit-learn 1.4 + XGBoost 2.0	Experiment tracking, model registry, drift detection
Bayesian A/B	PyMC3 (or PyMC 5) + ArviZ	Posterior sampling, ROPE, HDI
Multi-armed bandit	Custom Python (scipy.stats.beta)	Thompson sampling for creative rotation
Causal impact	tf-causalimpact (or R CausalImpact via rpy2)	Geo-holdout lift estimation
Reverse ETL	Census (or Hightouch)	Warehouse → ad platforms sync
BI / dashboarding	Looker (or Tableau / Metabase)	QBR dashboards, cohort retention, FDI SPC charts
Workflow orchestration	Dagster (or Airflow)	dbt runs, ML retraining, reverse ETL syncs, exception-triggered alerts
Code & config	GitHub + dbt Cloud	Versioned dbt models, Python retrainer, RACI-as-code

Part 2 — ML Scoring Model Retraining (THE WOW ELEMENT)

2.1 Why This Replaces V1's Recalibration

V1 recalibrated the TS-10 scoring engine by hand-tuning the 8 component weights (SC-001 through SC-008) and 27 modifiers (12 positive + 15 negative) using prior-quarter funded-loan rate deltas. This had three failure modes:

1. **Hand-tuning is non-reproducible.** No version control on the calibration decisions. No audit trail. Compliance cannot answer "which model version produced this adverse-action notice?"
2. **Heuristic weights ignore feature interactions.** The $DSCR \times FICO \times LTV$ interaction (NP-04 SWR-009 stacking risk) cannot be encoded in additive weights. A linear model misses the conditional structure.
3. **No drift detection.** V1 had no mechanism to detect when the prior-quarter's feature distribution shifted from the current quarter's. A silent shift produces silent scoring degradation.

GODMODE replaces V1 recalibration with a **production ML pipeline** that:

- Loads prior-quarter funded-loan data from the warehouse (Part 9)
- Engineers features per TS-10 contract (preserving SC-001 → SC-008 semantics)
- Trains a logistic-regression baseline (interpretable, Reg B-friendly) AND an XGBoost production model (captures interactions)
- Evaluates with AUC, Brier score, calibration curve
- Detects feature drift via Kolmogorov-Smirnov test
- Logs all artifacts to MLflow (model registry, versioned)
- Deploys new model to shadow-mode for 2-week soak before promotion

2.2 Production Pipeline (full Python module)

```
"""
TS-10 Quarterly Retraining Pipeline
=====

Production ML pipeline for retraining the TS-10 approval-scoring engine on
prior-quarter funded-loan data. Replaces V1's heuristic recalibration.

Architecture:
- Load: warehouse extract of prior-quarter leads (DSCR product only)
- Engineer features: TS-10 SC-001..SC-008 contract preserved
- Train: logistic-regression baseline (interpretable) + XGBoost production
- Evaluate: AUC, Brier score, calibration curve, lift at decile
- Drift detection: Kolmogorov-Smirnov per feature + PSI (Population Stability Index)
- Log: MLflow experiment + model registry
- Deploy: shadow-mode for 2-week soak, then promotion gate

Compliance:
- Reg B §1002.5(b)(1): "rather not say" is mid-default, never penalized
- Reg B §1002.4: no protected-class proxies in feature set
- Reg B §1002.9: model version logged with every adverse-action notice

Usage:
python ts10_retrainer.py --quarter Q2_2026 --warehouse-env prod --promote-gate

from __future__ import annotations

import argparse
import json
import logging
import os
import sys
import warnings
from dataclasses import dataclass, field, asdict
from datetime import datetime, timezone
from typing import Any, Dict, List, Optional, Tuple

import joblib
import mlflow
import mlflow.sklearn
import mlflow.xgboost
import numpy as np
import pandas as pd
import xgboost as xgb
from mlflow.tracking import MlflowClient
from scipy.stats import ks_2samp
from sklearn.calibration import calibration_curve
```

```

from sklearn.ensemble import GradientBoostingClassifier
from sklearn.exceptions import ConvergenceWarning
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    brier_score_loss,
    classification_report,
    roc_auc_score,
)
from sklearn.model_selection import TimeSeriesSplit
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore", category=ConvergenceWarning)
warnings.filterwarnings("ignore", category=FutureWarning)

# -----
# Logging
# -----
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-7s | %(name)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("ts10.retrainer")

# -----
# Configuration
# -----
@dataclass(frozen=True)
class RetrainConfig:
    """Immutable configuration for a quarterly retraining run."""

    quarter: str # e.g., "Q2_2026"
    warehouse_schema: str = "dscr_prod"
    product_type: str = "DSCR"
    lookback_quarters: int = 2 # train on prior 2 quarters
    test_holdout_fraction: float = 0.20 # last 20% chronologically
    tscv_splits: int = 3
    random_state: int = 42
    mlflow_tracking_uri: str = "https://mlflow.dscr-lender.internal"
    mlflow_experiment_name: str = "/dscr/ts10_scoring"
    mlflow_registry_model_name: str = "ts10_scoring_engine"
    promote_gate: bool = True # require AUC no-regression
    min_auc_delta_vs_prod: float = -0.02 # max tolerated regression
    psi_drift_threshold: float = 0.20 # PSI > 0.20 = significant drift
    ks_pvalue_threshold: float = 0.05 # p < 0.05 = drift detected

    # Feature contract — mirrors TS-10 SC-001..SC-008 + modifiers
    features: Tuple[str, ...] = (
        "dscr",
        "fico",
        "ltv",
        "reserves_months",
        "property_type_score",
        "doc_readiness_score",
        "experience_doors",
        "edge_case_indicator",
        "llc_structure",
        "lease_in_place",
        "str_permissive_market",
        "fn_with_us_llc",
        "prior_credit_event_seasoning_months",
    )
    target: str = "funded_loan"

```

```

# XGBoost hyperparameters (start; Optuna can tune)
xgb_params: Dict[str, Any] = field(default_factory=lambda: {
    "objective": "binary:logistic",
    "eval_metric": ["auc", "logloss"],
    "max_depth": 4,
    "learning_rate": 0.05,
    "subsample": 0.8,
    "colsample_bytree": 0.8,
    "min_child_weight": 5,
    "reg_alpha": 0.1,
    "reg_lambda": 1.0,
    "random_state": 42,
})
xgb_num_boost_round: int = 500
xgb_early_stopping_rounds: int = 20

# Logistic-regression hyperparameters
lr_params: Dict[str, Any] = field(default_factory=lambda: {
    "penalty": "l2",
    "C": 1.0,
    "solver": "lbfgs",
    "max_iter": 1000,
    "random_state": 42,
})

# -----
# Property-type scoring rubric (mirrors TS-10 SC-005)
# -----
PROPERTY_TYPE_SCORE: Dict[str, int] = {
    "SFR": 10,
    "2-4_unit": 9,
    "condo_warrantable": 8,
    "adu_permitted": 8,
    "condo_non_warrantable": 3,
    "condotel": 0,
}

# -----
# Retrainer
# -----
class TS10Retrainer:
    """Quarterly retraining pipeline for the TS-10 scoring engine.

    The pipeline is deterministic given the same input data and random_state.
    All artifacts (model, scaler, feature importance, drift report, calibration
    data) are logged to MLflow for full reproducibility and audit.
    """

    def __init__(self, config: RetrainConfig, warehouse_conn) -> None:
        self.cfg = config
        self.conn = warehouse_conn
        self.models: Dict[str, Any] = {}
        self.metrics: Dict[str, Any] = {}
        self.scaler: Optional[StandardScaler] = None

        mlflow.set_tracking_uri(self.cfg.mlflow_tracking_uri)
        mlflow.set_experiment(self.cfg.mlflow_experiment_name)

# -----
# Data loading
# -----
def load_data(self) -> pd.DataFrame:
    """Load prior-quarter cohort data from CRM data warehouse.

```

```

Pulls DSCR leads created in the lookback window. funded_date IS NOT NULL
is the positive class target. The query is parameterized to prevent SQL
injection (warehouse_conn must be a SQLAlchemy engine or DBAPI2 conn).
"""
query = """
    SELECT
        l.lead_id,
        l.created_date,
        l.funded_date,
        l.dscr_at_application      AS dscr,
        l.fico_at_application      AS fico,
        l.ltv_at_application      AS ltv,
        l.reserves_months,
        l.property_type,
        l.doc_readiness_score,
        l.borrower_experience_doors,
        l.edge_case_indicators,
        l.entity_structure,
        l.lease_in_place,
        l.str_market_permissive,
        l.fn_with_us_llc,
        l.prior_credit_event_seasoning_months,
        CASE WHEN l.funded_date IS NOT NULL THEN 1 ELSE 0 END AS funded_loan
    FROM %(schema)s.fact_leads l
    WHERE l.product_type = %(product)s
        AND l.created_date >= DATEADD(quarter, -%(lookback)s, CURRENT_DATE)
        AND l.created_date < DATEADD(quarter, -1, CURRENT_DATE)
        AND l.dscr_at_application IS NOT NULL
        AND l.fico_at_application IS NOT NULL
        AND l.ltv_at_application IS NOT NULL
"""
params = {
    "schema": self.cfg.warehouse_schema,
    "product": self.cfg.product_type,
    "lookback": self.cfg.lookback_quarters,
}
df = pd.read_sql(
    query,
    con=self.conn,
    params=params,
    parse_dates=["created_date", "funded_date"],
)
logger.info("Loaded %d leads from prior %d quarters", len(df), self.cfg.lookback_quarters)
if len(df) < 500:
    raise RuntimeError(
        f"Insufficient training data: {len(df)} leads (minimum 500 required for stable retrain). "
        "Extend lookback_quarters or investigate warehouse extract."
    )
return df.sort_values("created_date").reset_index(drop=True)

# -----
# Feature engineering
# -----
def engineer_features(self, df: pd.DataFrame) -> pd.DataFrame:
    """Engineer features per TS-10 SC-001..SC-008 contract.

    Compliance: no protected-class proxies. property_type_score, llc_structure,
    edge_case_indicator are all credit-risk / underwriting factors per
    Reg B §1002.2(p).
    """
    df = df.copy()

    # SC-005 property-type score
    df["property_type_score"] = (
        df["property_type"].map(PROPERTY_TYPE_SCORE).fillna(0).astype(int)

```

```

)

# SC-006 doc-readiness score (already on 0-10 scale from FF-08 Q-009)
df["doc_readiness_score"] = df["doc_readiness_score"].fillna(5).astype(float)

# SC-007 experience band + doors
df["experience_doors"] = df["borrower_experience_doors"].fillna(0).astype(int)

# SC-008 edge-case indicator (composite from EG-06 list)
df["edge_case_indicator"] = (
    df["edge_case_indicators"]
    .apply(lambda x: 1 if (isinstance(x, (list, str)) and x) else 0)
    .astype(int)
)

# SC-001..SC-004 numeric casts
df["dscr"] = pd.to_numeric(df["dscr"], errors="coerce")
df["fico"] = pd.to_numeric(df["fico"], errors="coerce")
df["ltv"] = pd.to_numeric(df["ltv"], errors="coerce")
df["reserves_months"] = pd.to_numeric(df["reserves_months"], errors="coerce").fillna(0)

# LLC structure flag (entity vesting – universal accelerator)
df["llc_structure"] = (df["entity_structure"] == "LLC").astype(int)

# Lease-in-place flag (AP-001 accelerant)
df["lease_in_place"] = df["lease_in_place"].fillna(False).astype(int)

# STR-permissive market flag (GS-07 geo tier T1/T2 STR markets)
df["str_permissive_market"] = df["str_market_permissive"].fillna(False).astype(int)

# FN with US LLC (foreign-national accelerator)
df["fn_with_us_llc"] = df["fn_with_us_llc"].fillna(False).astype(int)

# Prior credit-event seasoning months (NP-009 / FP-001)
df["prior_credit_event_seasoning_months"] = (
    pd.to_numeric(df["prior_credit_event_seasoning_months"], errors="coerce")
    .fillna(999) # 999 = no prior event (effectively ∞ seasoning)
)

# Drop rows with NaN in core features after engineering
required = list(self.cfg.features) + [self.cfg.target]
df_out = df[required].dropna(subset=list(self.cfg.features))
if len(df_out) < len(df) - 0.05 * len(df):
    logger.warning(
        "Dropped %d rows during feature engineering (%.1f%% of cohort)",
        len(df) - len(df_out), 100 * (len(df) - len(df_out)) / max(len(df), 1),
    )
logger.info("Feature-engineered cohort: %d rows, %d features", len(df_out), len(self.cfg.features))
return df_out.reset_index(drop=True)

# -----
# Train baseline logistic regression (interpretable)
# -----
def train_baseline_logistic(
    self,
    X_train: pd.DataFrame,
    y_train: pd.Series,
) -> Tuple[LogisticRegression, StandardScaler, pd.DataFrame]:
    """Train interpretable logistic regression as baseline + Reg B-safe fallback.

    Returns (model, scaler, feature_importance_df).
    """
    self.scaler = StandardScaler()
    X_train_scaled = self.scaler.fit_transform(X_train)

```

```

model = LogisticRegression(**self.cfg.lr_params)
model.fit(X_train_scaled, y_train)

importance = (
    pd.DataFrame({
        "feature": list(self.cfg.features),
        "coefficient": model.coef_[0],
    })
    .assign(abs_coeff=lambda d: d["coefficient"].abs())
    .sort_values("abs_coeff", ascending=False)
    .drop(columns="abs_coeff")
    .reset_index(drop=True)
)
logger.info("Logistic regression feature importance:\n%s", importance.to_string(index=False))
return model, self.scaler, importance

# -----
# Train XGBoost production model
# -----
def train_xgboost(
    self,
    X_train: pd.DataFrame,
    y_train: pd.Series,
    X_val: pd.DataFrame,
    y_val: pd.Series,
) -> xgb.Booster:
    """Train XGBoost for production scoring. Captures non-linear interactions
    (DSCR×FICO×LTV stacking risk) that linear logistic regression misses."""
    dtrain = xgb.DMatrix(X_train, label=y_train, feature_names=list(self.cfg.features))
    dval = xgb.DMatrix(X_val, label=y_val, feature_names=list(self.cfg.features))

    model = xgb.train(
        self.cfg.xgb_params,
        dtrain,
        num_boost_round=self.cfg.xgb_num_boost_round,
        evals=[(dtrain, "train"), (dval, "val")],
        early_stopping_rounds=self.cfg.xgb_early_stopping_rounds,
        verbose_eval=50,
    )
    logger.info(
        "XGBoost trained. Best iteration: %d, val AUC: %.4f",
        model.best_iteration, float(model.best_score),
    )
    return model

# -----
# Evaluate
# -----
def evaluate(
    self,
    model: Any,
    X_test: pd.DataFrame,
    y_test: pd.Series,
    model_type: str,
    scaler: Optional[StandardScaler] = None,
) -> Dict[str, Any]:
    """Evaluate model with AUC, Brier, calibration curve, decile lift."""
    if model_type == "xgboost":
        dtest = xgb.DMatrix(X_test, feature_names=list(self.cfg.features))
        y_pred_proba = model.predict(dtest)
    elif model_type == "logistic":
        assert scaler is not None, "Logistic regression requires scaler"
        y_pred_proba = model.predict_proba(scaler.transform(X_test))[:, 1]
    elif model_type == "gradient_boosting":
        y_pred_proba = model.predict_proba(X_test)[:, 1]

```

```

else:
    raise ValueError(f"Unknown model_type: {model_type}")

auc = float(roc_auc_score(y_test, y_pred_proba))
brier = float(brier_score_loss(y_test, y_pred_proba))

# Calibration curve (10 bins)
frac_pos, mean_pred = calibration_curve(y_test, y_pred_proba, n_bins=10, strategy="quantile")
calibration = [
    {"mean_predicted": float(p), "fraction_positive": float(f)}
    for p, f in zip(mean_pred, frac_pos)
]

# Decile lift (top decile vs base rate)
decile_df = pd.DataFrame({"y": y_test.values, "p": y_pred_proba})
decile_df["decile"] = pd.qcut(decile_df["p"], 10, labels=False, duplicates="drop")
base_rate = float(decile_df["y"].mean())
top_decile_rate = float(decile_df.loc[decile_df["decile"] == decile_df["decile"].max(), "y"].mean())
lift_top_decile = top_decile_rate / base_rate if base_rate > 0 else 0.0

# Classification report at 0.5 threshold (informational only – production
# uses tier-routing thresholds from TS-10 Part 1E, not 0.5)
y_pred_label = (y_pred_proba >= 0.5).astype(int)
clf_report = classification_report(y_test, y_pred_label, output_dict=True)

self.metrics[model_type] = {
    "auc": auc,
    "brier_score": brier,
    "lift_top_decile": lift_top_decile,
    "base_rate": base_rate,
    "calibration": calibration,
    "classification_report": clf_report,
}
logger.info(
    "%s – AUC: %.4f | Brier: %.4f | Lift@10: %.2fx | Base rate: %.3f",
    model_type, auc, brier, lift_top_decile, base_rate,
)
return self.metrics[model_type]

# -----
# Drift detection (Kolmogorov-Smirnov + Population Stability Index)
# -----
def detect_drift(
    self,
    X_train: pd.DataFrame,
    X_test: pd.DataFrame,
) -> Dict[str, Dict[str, Any]]:
    """Detect feature drift between training and test sets.

    Two complementary statistics:
    - Kolmogorov-Smirnov: p < 0.05 → drift detected (per-feature)
    - Population Stability Index: PSI > 0.20 → significant drift
      (industry-standard threshold; PSI < 0.10 = stable, 0.10-0.20 = monitor,
      > 0.20 = action required)
    """
    drift: Dict[str, Dict[str, Any]] = {}
    for col in self.cfg.features:
        ks_stat, ks_p = ks_2samp(X_train[col], X_test[col])
        psi = self._psi(X_train[col], X_test[col])
        drifted = (ks_p < self.cfg.ks_pvalue_threshold) or (psi > self.cfg.psi_drift_threshold)
        drift[col] = {
            "ks_statistic": float(ks_stat),
            "ks_p_value": float(ks_p),
            "psi": float(psi),
            "drift_detected": bool(drifted),
        }

```

```

    }
    if drifted:
        logger.warning(
            "DRIFT in %s - KS=%.4f (p=%.4f), PSI=%.4f",
            col, ks_stat, ks_p, psi,
        )
    self.metrics["drift"] = drift
    return drift

@staticmethod
def _psi(expected: pd.Series, actual: pd.Series, bins: int = 10) -> float:
    """Population Stability Index. PSI < 0.10 stable, > 0.20 action required."""
    eps = 1e-4
    bins_edges = np.linspace(min(expected.min(), actual.min()),
                              max(expected.max(), actual.max()), bins + 1)
    exp_pct = np.histogram(expected, bins=bins_edges)[0] / len(expected)
    act_pct = np.histogram(actual, bins=bins_edges)[0] / len(actual)
    exp_pct = np.clip(exp_pct, eps, None)
    act_pct = np.clip(act_pct, eps, None)
    return float(np.sum((act_pct - exp_pct) * np.log(act_pct / exp_pct)))

# -----
# No-regression promotion gate
# -----
def check_promotion_gate(self) -> Tuple[bool, str]:
    """Compare new XGBoost AUC vs current production model AUC.

    Returns (promote: bool, rationale: str).
    """
    if not self.cfg.promote_gate:
        return True, "Promotion gate disabled in config"
    client = MlflowClient()
    try:
        prod_versions = client.get_latest_versions(
            self.cfg.mlflow_registry_model_name, stages=["Production"]
        )
        if not prod_versions:
            return True, "No prior production version - first deploy"
        prod_run_id = prod_versions[0].run_id
        prod_auc = client.get_metric_history(prod_run_id, "xgboost_auc")[-1].value
        new_auc = self.metrics["xgboost"]["auc"]
        delta = new_auc - prod_auc
        if delta < self.cfg.min_auc_delta_vs_prod:
            return False, (
                f"REGRESSION: new AUC {new_auc:.4f} < prod AUC {prod_auc:.4f} "
                f"by {abs(delta):.4f} (threshold {abs(self.cfg.min_auc_delta_vs_prod):.4f}). "
                f"Auto-promotion blocked. Investigate data quality, drift report, or feature engineering."
            )
        return True, f"PASS: new AUC {new_auc:.4f} >= prod AUC {prod_auc:.4f} + threshold"
    except Exception as exc:
        logger.exception("Promotion gate check failed: %s", exc)
        return False, f"Promotion gate check errored: {exc}"

# -----
# Full retraining pipeline
# -----
def retrain(self) -> Dict[str, Any]:
    """Full retraining pipeline. Returns the metrics dict (also logged to MLflow)."""
    run_name = f"ts10_retrain_{self.cfg.quarter}_{datetime.now(timezone.utc).strftime('%Y%m%d_%H%M%S')}"
    with mlflow.start_run(run_name=run_name) as run:
        run_id = run.info.run_id
        logger.info("MLflow run started: %s", run_id)
        mlflow.log_params(asdict(self.cfg))
        mlflow.set_tag("quarter", self.cfg.quarter)
        mlflow.set_tag("pipeline", "ts10_quarterly_retrain")

```

```

mlflow.set_tag("compliance", "reg_b_section_1002.5_b_1_compliant")

# 1. Load + engineer
df = self.load_data()
df = self.engineer_features(df)

X = df[list(self.cfg.features)]
y = df[self.cfg.target]

# 2. Time-series cross-validation (no leakage – train on older, val on newer)
tscv = TimeSeriesSplit(n_splits=self.cfg.tscv_splits)
last_train_idx, last_val_idx = None, None
for train_idx, val_idx in tscv.split(X):
    last_train_idx, last_val_idx = train_idx, val_idx
assert last_train_idx is not None and last_val_idx is not None, "TSCV failed"
X_train, X_val = X.iloc[last_train_idx], X.iloc[last_val_idx]
y_train, y_val = y.iloc[last_train_idx], y.iloc[last_val_idx]

# 3. Final test set: last 20% chronologically (held out from all training)
split_idx = int(len(df) * (1 - self.cfg.test_holdout_fraction))
X_test, y_test = X.iloc[split_idx:], y.iloc[split_idx:]

logger.info(
    "Splits – train: %d | val: %d | test: %d | base rate (train): %.3f",
    len(X_train), len(X_val), len(X_test), float(y_train.mean()),
)

# 4. Train baseline logistic
log_model, scaler, log_importance = self.train_baseline_logistic(X_train, y_train)
self.models["logistic"] = log_model
self.evaluate(log_model, X_test, y_test, "logistic", scaler=scaler)

# 5. Train XGBoost
xgb_model = self.train_xgboost(X_train, y_train, X_val, y_val)
self.models["xgboost"] = xgb_model
self.evaluate(xgb_model, X_test, y_test, "xgboost")

# 6. Optional: GradientBoosting (sklearn) as backup if XGBoost unavailable
try:
    gb_model = GradientBoostingClassifier(
        n_estimators=200, max_depth=3, learning_rate=0.05,
        subsample=0.8, random_state=self.cfg.random_state,
    )
    gb_model.fit(X_train, y_train)
    self.models["gradient_boosting"] = gb_model
    self.evaluate(gb_model, X_test, y_test, "gradient_boosting")
except Exception as exc:
    logger.warning("GradientBoosting train failed: %s", exc)

# 7. Drift detection (train vs test)
drift_report = self.detect_drift(X_train, X_test)
n_drifted = sum(1 for v in drift_report.values() if v["drift_detected"])
mlflow.log_metric("n_features_drifted", n_drifted)
mlflow.log_dict(drift_report, "drift_report.json")
mlflow.log_dict(log_importance.to_dict(orient="records"), "logistic_feature_importance.json")

# 8. Log metrics to MLflow
mlflow.log_metrics({
    f"{k}_auc": v["auc"]
    for k, v in self.metrics.items()
    if isinstance(v, dict) and "auc" in v
})
mlflow.log_metrics({
    f"{k}_brier": v["brier_score"]
    for k, v in self.metrics.items()

```

```

        if isinstance(v, dict) and "brier_score" in v
    })
    mlflow.log_metric("train_base_rate", float(y_train.mean()))
    mlflow.log_metric("test_base_rate", float(y_test.mean()))
    mlflow.log_metric("cohort_size", int(len(df)))

# 9. Log models
mlflow.sklearn.log_model(log_model, "logistic_baseline")
mlflow.xgboost.log_model(xgb_model, "xgboost_production")
if "gradient_boosting" in self.models:
    mlflow.sklearn.log_model(self.models["gradient_boosting"], "gradient_boosting_backup")

# 10. Save scaler artifact
scaler_path = f"/tmp/scaler_{self.cfg.quarter}.pkl"
joblib.dump(scaler, scaler_path)
mlflow.log_artifact(scaler_path, artifact_path="scaler")

# 11. Promotion gate
promote, rationale = self.check_promotion_gate()
mlflow.set_tag("promotion_decision", "promote" if promote else "block")
mlflow.set_tag("promotion_rationale", rationale)
self.metrics["promotion"] = {"promote": promote, "rationale": rationale}

if promote:
    # Register to MLflow Model Registry as Staging (then promoted to Production after 2-week shadow soak)
    client = MlflowClient()
    try:
        client.create_registered_model(self.cfg.mlflow_registry_model_name)
    except Exception:
        pass # already exists
    xgb_uri = f"runs://{run_id}/xgboost_production"
    mv = client.create_model_version(
        name=self.cfg.mlflow_registry_model_name,
        source=xgb_uri,
        run_id=run_id,
        tags={"quarter": self.cfg.quarter, "stage": "staging_shadow"},
    )
    client.transition_model_version_stage(
        name=self.cfg.mlflow_registry_model_name,
        version=mv.version,
        stage="Staging",
        archive_existing_versions=False,
    )
    logger.info("XGBoost registered as version %d (Staging / shadow-mode)", mv.version)
    self.metrics["model_version"] = mv.version

# 12. Persist full metrics JSON for QBR deck
metrics_path = f"/tmp/ts10_metrics_{self.cfg.quarter}.json"
with open(metrics_path, "w") as f:
    json.dump(self.metrics, f, indent=2, default=str)
mlflow.log_artifact(metrics_path, artifact_path="metrics")

return self.metrics

# -----
# CLI entrypoint
# -----
def parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(description="TS-10 Quarterly Retrainer")
    p.add_argument("--quarter", required=True, help="Quarter label, e.g., Q2_2026")
    p.add_argument("--warehouse-env", default="prod", help="Warehouse connection env (prod/staging)")
    p.add_argument("--promote-gate", action="store_true", help="Enable no-regression promotion gate")
    p.add_argument("--lookback-quarters", type=int, default=2)
    return p.parse_args()

```

```
def get_warehouse_connection(env: str):
    """Return a SQLAlchemy engine for the warehouse. Replace with your org's
    connection helper (Snowflake / BigQuery / Redshift)."""
    from sqlalchemy import create_engine
    if env == "prod":
        return create_engine(os.environ["DSCR_WAREHOUSE_PROD_URL"], pool_pre_ping=True)
    return create_engine(os.environ["DSCR_WAREHOUSE_STAGING_URL"], pool_pre_ping=True)

if __name__ == "__main__":
    args = parse_args()
    cfg = RetrainConfig(
        quarter=args.quarter,
        lookback_quarters=args.lookback_quarters,
        promote_gate=args.promote_gate,
    )
    conn = get_warehouse_connection(args.warehouse_env)
    retrainer = TS10Retrainer(cfg, conn)
    metrics = retrainer.retrain()
    print(json.dumps({
        "quarter": cfg.quarter,
        "xgboost_auc": metrics.get("xgboost", {}).get("auc"),
        "logistic_auc": metrics.get("logistic", {}).get("auc"),
        "n_features_drifted": sum(
            1 for v in metrics.get("drift", {}).values() if v.get("drift_detected")
        ),
        "promotion": metrics.get("promotion"),
    }, indent=2, default=str))
    sys.exit(0 if metrics["promotion"]["promote"] else 1)
```

2.3 A/B Test Design — New Model vs Current Production Model

The promotion gate (§ 2.2 `check_promotion_gate`) blocks regressions but a model that **passes the gate** still must **beat the current production model in production traffic** before full rollout. This requires a controlled A/B test.

2.3.1 Shadow-mode soak (week 1–2 of Q+1)

- New model is deployed behind a feature flag (`ts10_model_version = "v_next"`).
- 10% of leads (random by `lead_id % 10 == 0`) are scored by **both** `v_current` and `v_next`.
- `v_current` scores drive routing (production). `v_next` scores are logged only.
- After 14 days, compute:
 - `v_next` vs `v_current` AUC on funded-loan outcomes (need `funded_date`, so this lags by ~21 days average close time)
 - Tier-routing delta: % of leads routed to TIER_A/B by `v_next` vs `v_current`
 - Calibration comparison (Brier score)

2.3.2 Bayesian A/B test (week 3–6 of Q+1)

After shadow soak, run a real traffic A/B test (Part 3 framework):

Arm	Model version	Traffic share	Optimization event
Control	v_current (prior quarter's production)	50%	Tier_Routed_A_or_B
Variant	v_next (this quarter's retrain)	50%	Tier_Routed_A_or_B

Decision rule (from Part 3 Bayesian A/B test):

- $p_{\text{variant_better}} > 0.95 \rightarrow$ promote v_next to Production in MLflow registry
- $p_{\text{control_better}} > 0.95 \rightarrow$ keep v_current, archive v_next, investigate
- $p_{\text{equivalent}} > 0.95 \rightarrow$ declare equivalence; promote v_next only if it's simpler / cheaper / more interpretable
- Otherwise $\rightarrow$ keep testing (extend to week 8)

2.3.3 Rollback plan

If v_next is promoted and a regression is detected in week 7–8 (e.g., funded-loan rate drops >5% relative to v_current baseline), execute rollback:

1. Set `ts10_model_version = "v_current"` in feature flag (instant rollback, no redeploy).
2. Transition MLflow model version v_next from `Production` $\rightarrow$ `Archived`.
3. File an incident in `fact_iteration_exceptions` (Part 1 § 1.5).
4. Schedule expedited QIC within 5 business days to investigate root cause.

2.4 Feature Importance Reporting (Reg B Compliance)

The logistic regression baseline provides interpretable coefficients. Per Reg B §1002.9 adverse-action notice requirements, when a lead is routed to TIER_D (decline/defer), the adverse-action notice must cite the **principal reasons** for the adverse decision. The logistic-regression feature importance (`§ 2.2 train_baseline_logistic  $\rightarrow$  importance DataFrame`) feeds the adverse-action notice generator:

```
# Adverse-action reason generator (uses logistic baseline coefficients for interpretability)
def generate_adverse_action_reasons(
    lead_features: dict,
    logistic_model: LogisticRegression,
    scaler: StandardScaler,
    feature_names: list,
    top_n: int = 3,
) -> list[str]:
    """Return top-N principal reasons for adverse decision, per Reg B §1002.9.

    Uses Shapley-style contribution: coefficient * (scaled_value - mean).
    Compliance-tested: outputs neutral credit-risk language, never protected-class
    language. Reviewed by ECOA counsel annually.
    """
    import numpy as np
    x = np.array([[lead_features[f] for f in feature_names]])
    x_scaled = scaler.transform(x)
    contributions = x_scaled[0] * logistic_model.coef_[0]
    ranked = sorted(zip(feature_names, contributions), key=lambda kv: kv[1])
```

```

reasons = []
REASON_MAP = {
    "dscr": "Debt service coverage ratio below program minimum",
    "fico": "Credit score below program minimum",
    "ltv": "Loan-to-value ratio exceeds program maximum",
    "reserves_months": "Reserves below program minimum",
    "property_type_score": "Property type not eligible for program",
    "doc_readiness_score": "Documentation insufficient for underwriting",
    "experience_doors": "Investment experience below program threshold",
    "edge_case_indicator": "Edge-case indicators require specialty-lender review",
    "prior_credit_event_seasoning_months": "Prior credit event outside seasoning window",
}
for feat, _ in ranked[:top_n]:
    if feat in REASON_MAP and lead_features.get(feat, 0) != 0:
        reasons.append(REASON_MAP[feat])
return reasons or ["Application does not meet program minimums - see specialty-lender routing options"]

```

2.5 Retraining Schedule (Annual Calendar)

Quarter	Retrain trigger	Special notes
Q1 (Jan)	First QIC of year	Full retrain on Q3+Q4 prior-year data; promote-gate active
Q2 (Apr)	Annual swarm refresh	Full retrain + EG-06 / GS-07 / FF-08 / AC-09 refresh; longest cycle (20 days)
Q3 (Jul)	Standard QIC	Full retrain; check drift report for seasonal STR-market shift
Q4 (Oct)	Standard QIC	Full retrain; year-end LTV/FICO band check before holiday slowdown

Part 3 — Bayesian A/B Testing Framework (PyMC3)

3.1 Why Bayesian (Not Frequentist) for the Swarm

V1 specified a 9-test A/B calendar with frequentist significance testing ($p < 0.05$ after $n \geq 1,000$ per arm). This had four problems:

1. **Fixed sample size is unrealistic.** DSCR funded-loan rate is ~5–8% of leads. Detecting a 0.5pt lift at 80% power requires ~3,500 leads per arm — at 200 leads/day, that's 35 days per test, longer than the mid-quarter stability window.
2. **No peeking.** Frequentist tests require pre-registered sample size; peeking inflates false-positive rate. Marketing-ops cannot peek at A/B tests mid-quarter — operationally unacceptable.
3. **No "no-difference" conclusion.** Frequentist tests can only reject or fail to reject the null. They cannot declare "these variants are practically equivalent" — which is what marketing-ops needs to stop a test early and free traffic for the next test.

4. **No prior.** V1 ignored prior-quarter knowledge. Each A/B test started from scratch. Bayesian tests incorporate prior-quarter conversion rates as informative priors, shrinking the sample size required.

GODMODE uses PyMC3 Bayesian A/B tests with **Beta-Binomial conjugate models** (fast, exact), **Region of Practical Equivalence (ROPE)** decisions, and **Highest Density Intervals (HDI)** for uncertainty quantification. The framework supports both Binomial (conversion / no conversion) and Beta-Binomial (rate metrics like CTR).

3.2 Production Framework

```
"""
Bayesian A/B Test Framework for DSCR Swarm
=====

Replaces V1's frequentist A/B testing with PyMC3 Bayesian framework:
- Beta-Binomial conjugate model (fast, exact for conversion metrics)
- Region of Practical Equivalence (ROPE) for "no difference" decisions
- Highest Density Interval (HDI) for uncertainty quantification
- Sequential testing (peek any time without inflating false positives)
- Informative priors from prior-quarter data

Decision rules (configurable per test):
- p_variant_better > 0.95 → DECLARE VARIANT WINNER
- p_control_better > 0.95 → DECLARE VARIANT LOSER
- p_equivalent > 0.95 → DECLARE EQUIVALENCE (stop test, no difference)
- Otherwise → KEEP TESTING

Compliance: Decision rules documented per test in the A/B test registry
(audit trail for Reg B / state UDAAP review).
"""

from __future__ import annotations

import json
import logging
import warnings
from dataclasses import dataclass, field, asdict
from datetime import datetime, timezone
from typing import Any, Dict, List, Literal, Optional, Tuple

import arviz as az
import numpy as np
import pandas as pd
import pymc3 as pm

warnings.filterwarnings("ignore", category=FutureWarning)
logger = logging.getLogger("dscr.bayesian_ab")

@dataclass(frozen=True)
class ABTestConfig:
    """Configuration for a single Bayesian A/B test."""
    test_id: str # e.g., "ABT-2026-Q2-001"
    test_name: str # human-readable
    metric: str # "funded_loan_rate" / "tier_a_b_rate" / "ctr"
    metric_type: Literal["binomial", "rate"] = "binomial"
    # Priors - Beta(α, β). Default Beta(1,1) = uniform (uninformative).
    # For informative priors, set α = prior_conversions + 1, β = prior_visitors - prior_conversions + 1.
    prior_alpha: int = 1
```

```

prior_beta: int = 1
# ROPE – Region of Practical Equivalence. Effect sizes within ROPE = "no difference".
# For funded-loan rate (base ~5%), ROPE = ±0.5pt = (-0.005, +0.005).
# For CTR (base ~1%), ROPE = ±0.1pt = (-0.001, +0.001).
rope: Tuple[float, float] = (-0.005, 0.005)
# Decision threshold
decision_threshold: float = 0.95
# MCMC sampling
samples: int = 10_000
tune: int = 5_000
chains: int = 4
cores: int = 4
# Target – when do we stop? Either n_per_arm reached or posterior decisive
target_n_per_arm: int = 5_000
early_stop_on_decisive: bool = True

# -----
# Beta-Binomial Bayesian A/B Test (conversion metrics)
# -----
class BayesianABTest:
    """Bayesian A/B test using Beta-Binomial model.

    Closed-form posterior:  $\text{Beta}(\alpha + \text{conversions}, \beta + \text{visitors} - \text{conversions})$ .
    PyMC3 is used for the effect-size distribution and HDI computation, but the
    marginal posteriors are exact (no MCMC needed for the marginals themselves).
    """

    def __init__(self, config: ABTestConfig) -> None:
        self.cfg = config
        self.trace: Optional[az.InferenceData] = None
        self.results: Dict[str, Any] = {}

    def run_test(
        self,
        control_conversions: int,
        control_visitors: int,
        variant_conversions: int,
        variant_visitors: int,
    ) -> Dict[str, Any]:
        """Run Bayesian A/B test.

        Args:
            control_conversions: int, conversions in control arm
            control_visitors: int, total visitors (denominator) in control arm
            variant_conversions: int, conversions in variant arm
            variant_visitors: int, total visitors in variant arm

        Returns:
            dict with probabilities, effect size mean + HDI, decision, and
            raw posterior samples for downstream visualization.
        """
        if control_conversions > control_visitors or variant_conversions > variant_visitors:
            raise ValueError("Conversions cannot exceed visitors")
        if control_visitors < 10 or variant_visitors < 10:
            logger.warning("Very small sample – results will be dominated by prior")

        with pm.Model() as model:
            # Priors
            p_control = pm.Beta(
                "p_control",
                alpha=self.cfg.prior_alpha,
                beta=self.cfg.prior_beta,
            )
            p_variant = pm.Beta(

```

```

        "p_variant",
        alpha=self.cfg.prior_alpha,
        beta=self.cfg.prior_beta,
    )

    # Likelihoods
    pm.Binomial(
        "obs_control",
        n=control_visitors,
        p=p_control,
        observed=control_conversions,
    )
    pm.Binomial(
        "obs_variant",
        n=variant_visitors,
        p=p_variant,
        observed=variant_conversions,
    )

    # Effect size (variant - control) - the decision variable
    pm.Deterministic("effect", p_variant - p_control)
    # Relative lift (variant / control - 1) - for reporting
    pm.Deterministic("relative_lift", p_variant / p_control - 1.0)

    # Sample posterior
    self.trace = pm.sample(
        draws=self.cfg.samples,
        tune=self.cfg.tune,
        chains=self.cfg.chains,
        cores=self.cfg.cores,
        target_accept=0.95,
        return_inferencedata=True,
        progressbar=False,
    )

    # Compute decision probabilities
    effect_samples = (
        self.trace.posterior["effect"]
        .values.reshape(-1) # flatten chains x draws
    )
    lift_samples = (
        self.trace.posterior["relative_lift"]
        .values.reshape(-1)
    )
    rope_lo, rope_hi = self.cfg.rope
    p_variant_better = float(np.mean(effect_samples > rope_hi))
    p_control_better = float(np.mean(effect_samples < rope_lo))
    p_equivalent = float(np.mean((effect_samples >= rope_lo) & (effect_samples <= rope_hi)))

    # HDI (Highest Density Interval) - 95% by default
    hdi = az.hdi(effect_samples, hdi_prob=0.95)
    effect_mean = float(np.mean(effect_samples))
    lift_mean = float(np.mean(lift_samples))

    # Decision
    decision, decision_reason = self._decide(
        p_variant_better, p_control_better, p_equivalent
    )

    # Base rates (observed)
    control_rate = control_conversions / control_visitors if control_visitors > 0 else 0.0
    variant_rate = variant_conversions / variant_visitors if variant_visitors > 0 else 0.0

    self.results = {
        "test_id": self.cfg.test_id,

```

```

        "test_name": self.cfg.test_name,
        "metric": self.cfg.metric,
        "control": {
            "conversions": control_conversions,
            "visitors": control_visitors,
            "rate": control_rate,
        },
        "variant": {
            "conversions": variant_conversions,
            "visitors": variant_visitors,
            "rate": variant_rate,
        },
        "p_variant_better": p_variant_better,
        "p_control_better": p_control_better,
        "p_equivalent": p_equivalent,
        "effect_size_mean": effect_mean,
        "effect_size_hdi_95": [float(hdi[0]), float(hdi[1])],
        "relative_lift_mean": lift_mean,
        "decision": decision,
        "decision_reason": decision_reason,
        "rope": list(self.cfg.rope),
        "decision_threshold": self.cfg.decision_threshold,
        "samples_total": len(effect_samples),
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
    }
    logger.info(
        "A/B test %s - %s | p_variant_better=%3f | p_control_better=%3f | p_equiv=%3f | decision=%s",
        self.cfg.test_id, self.cfg.test_name,
        p_variant_better, p_control_better, p_equivalent, decision,
    )
    return self.results

def _decide(
    self,
    p_variant_better: float,
    p_control_better: float,
    p_equivalent: float,
) -> Tuple[str, str]:
    """Apply decision rule. Returns (decision, reason)."""
    t = self.cfg.decision_threshold
    if p_variant_better > t:
        return (
            "DECLARE_VARIANT_WINNER",
            f"P(variant better) = {p_variant_better:.3f} > {t}. Promote variant to 100% traffic.",
        )
    if p_control_better > t:
        return (
            "DECLARE_VARIANT_LOSER",
            f"P(control better) = {p_control_better:.3f} > {t}. Stop variant; keep control.",
        )
    if p_equivalent > t:
        return (
            "DECLARE_EQUIVALENCE",
            f"P(equivalent within ROPE {self.cfg.rope}) = {p_equivalent:.3f} > {t}. "
            f"Stop test; promote variant only if simpler/cheaper.",
        )
    return (
        "KEEP_TESTING",
        f"No decisive probability > {t}. Continue data collection.",
    )

# -----
# Sample-size estimator (informative prior)
# -----

```

```

def estimate_min_sample_size(
    base_rate: float,
    mde: float,
    decision_threshold: float = 0.95,
    rope_half_width: Optional[float] = None,
) -> Dict[str, int]:
    """Estimate min sample size per arm for a Bayesian A/B test.

    Uses normal approximation for expected posterior variance:
    Var(p̂) ≈ p(1-p)/n
    For decisive result, need |effect| / sqrt(2*Var) > z where z corresponds to
    decision_threshold (e.g., z=1.645 for 0.95 one-sided).

    Args:
        base_rate: expected conversion rate in control (e.g., 0.05 for 5%)
        mde: minimum detectable effect in absolute terms (e.g., 0.005 for 0.5pt)
        decision_threshold: posterior probability threshold (default 0.95)
        rope_half_width: half-width of ROPE. If None, set = mde/2.

    Returns:
        dict with n_per_arm, n_total, and assumptions used.
    """
    from scipy.stats import norm
    z = norm.ppf(decision_threshold)
    rope_half_width = rope_half_width if rope_half_width is not None else mde / 2
    # Approximate: need |effect| - rope_half_width > z * sqrt(2 * p*(1-p)/n)
    # Solve for n:
    n = int(np.ceil((2 * base_rate * (1 - base_rate) * z**2) / (mde - rope_half_width) ** 2))
    return {
        "n_per_arm": n,
        "n_total": 2 * n,
        "base_rate": base_rate,
        "mde_absolute": mde,
        "mde_relative": mde / base_rate,
        "decision_threshold": decision_threshold,
        "rope_half_width": rope_half_width,
        "method": "normal_approximation",
    }

# -----
# Sequential test runner (peek-safe - Bayesian tests have no peeking penalty)
# -----
class SequentialABTestRunner:
    """Peek-safe sequential A/B test runner.

    Bayesian tests have no false-positive inflation from peeking. This runner
    checks the test daily and stops when a decisive posterior is reached,
    freeing traffic for the next test.
    """

    def __init__(self, config: ABTestConfig) -> None:
        self.cfg = config
        self.ab = BayesianABTest(config)
        self.history: List[Dict[str, Any]] = []

    def daily_check(
        self,
        control_conversions: int,
        control_visitors: int,
        variant_conversions: int,
        variant_visitors: int,
    ) -> Dict[str, Any]:
        """Run a single daily check. Returns results + early-stop flag."""
        results = self.ab.run_test(

```

```

        control_conversions, control_visitors,
        variant_conversions, variant_visitors,
    )
    results["day_index"] = len(self.history)
    self.history.append(results)
    if self.cfg.early_stop_on_decisive and results["decision"] != "KEEP_TESTING":
        logger.info(
            "Early stop on day %d - decision: %s",
            len(self.history), results["decision"],
        )
    return results

@property
def should_stop(self) -> bool:
    if not self.history:
        return False
    last = self.history[-1]
    if last["decision"] != "KEEP_TESTING":
        return True
    if (last["control"]["visitors"] >= self.cfg.target_n_per_arm
        and last["variant"]["visitors"] >= self.cfg.target_n_per_arm):
        return True
    return False

# -----
# A/B Test Registry (audit trail)
# -----
class ABTestRegistry:
    """Registry of all A/B tests – audit trail for Reg B / state UDAAP review.

    Each test logs: hypothesis, prior, ROPE, decision threshold, observed data,
    decision, timestamp, sign-offs. Stored in warehouse table `fact_ab_tests`.
    """

    SCHEMA_DDL = """
CREATE TABLE IF NOT EXISTS dscr_prod.fact_ab_tests (
    test_id          VARCHAR(64)    PRIMARY KEY,
    test_name        TEXT           NOT NULL,
    metric           VARCHAR(64)    NOT NULL,
    metric_type      VARCHAR(16)    NOT NULL,
    prior_alpha      INT            NOT NULL,
    prior_beta       INT            NOT NULL,
    rope             NUMERIC(8,4)[] NOT NULL,
    decision_threshold NUMERIC(4,3) NOT NULL,
    control_conversions INT         NOT NULL,
    control_visitors  INT           NOT NULL,
    variant_conversions INT         NOT NULL,
    variant_visitors  INT           NOT NULL,
    p_variant_better  NUMERIC(6,4),
    p_control_better  NUMERIC(6,4),
    p_equivalent      NUMERIC(6,4),
    effect_size_mean  NUMERIC(8,5),
    effect_size_hdi_95 NUMERIC(8,5) [],
    decision          VARCHAR(32),
    decision_reason   TEXT,
    timestamp_utc     TIMESTAMPTZ  NOT NULL,
    signoffs_json     JSONB,
    raw_results_json  JSONB
);
"""

    def __init__(self, warehouse_conn) -> None:
        self.conn = warehouse_conn

```

```

def register(self, results: Dict[str, Any], signoffs: List[Dict[str, str]]) -> None:
    """Register test results + sign-offs to warehouse."""
    import sqlalchemy
    row = {
        **results,
        "signoffs_json": json.dumps(signoffs),
        "raw_results_json": json.dumps(results, default=str),
    }
    df = pd.DataFrame([row])
    df.to_sql("fact_ab_tests", self.conn, schema="dscr_prod",
             if_exists="append", index=False)
    logger.info("Registered A/B test %s to warehouse", results["test_id"])

# -----
# Example: Q2 2026 A/B test calendar (6 tests)
# -----
EXAMPLE_Q2_2026_CALENDAR = [
    {
        "test_id": "ABT-2026-Q2-001",
        "test_name": "SA-001 PI-1 vs PI-2 pattern-interrupt hook",
        "metric": "tier_a_b_rate",
        "rope": [-0.005, 0.005],
        "expected_base_rate": 0.42,
        "mde": 0.03,
    },
    {
        "test_id": "ABT-2026-Q2-002",
        "test_name": "Lead-magnet CTA: calculator vs decline-letter audit",
        "metric": "form_complete_rate",
        "rope": [-0.01, 0.01],
        "expected_base_rate": 0.18,
        "mde": 0.02,
    },
    {
        "test_id": "ABT-2026-Q2-003",
        "test_name": "Landing page: 3-case-study proof stack vs single-case",
        "metric": "funded_loan_rate",
        "rope": [-0.005, 0.005],
        "expected_base_rate": 0.05,
        "mde": 0.008,
    },
    {
        "test_id": "ABT-2026-Q2-004",
        "test_name": "Risk reversal: free-prequal vs $500-credit-guarantee",
        "metric": "tier_a_b_rate",
        "rope": [-0.005, 0.005],
        "expected_base_rate": 0.42,
        "mde": 0.025,
    },
    {
        "test_id": "ABT-2026-Q2-005",
        "test_name": "Meta ad set: customer-file lookalike 1% vs 3%",
        "metric": "tier_a_b_rate",
        "rope": [-0.005, 0.005],
        "expected_base_rate": 0.42,
        "mde": 0.02,
    },
    {
        "test_id": "ABT-2026-Q2-006",
        "test_name": "TS-10 model v_current vs v_next (shadow-mode promotion)",
        "metric": "funded_loan_rate",
        "rope": [-0.003, 0.003],
        "expected_base_rate": 0.05,
        "mde": 0.005,
    },
]

```

```

    },
]

# -----
# CLI / example entrypoint
# -----
if __name__ == "__main__":
    cfg = ABTestConfig(
        test_id="ABT-2026-Q2-001",
        test_name="SA-001 PI-1 vs PI-2 hook",
        metric="tier_a_b_rate",
        rope=(-0.005, 0.005),
        prior_alpha=43,      # informative prior: 42 prior-quarter conversions / 100 leads
        prior_beta=58,
    )
    ab = BayesianABTest(cfg)
    results = ab.run_test(
        control_conversions=423, control_visitors=1000,
        variant_conversions=498, variant_visitors=1000,
    )
    print(json.dumps(results, indent=2, default=str))

# Sample size estimator
n = estimate_min_sample_size(base_rate=0.42, mde=0.03)
print("\nSample size estimate:", json.dumps(n, indent=2))

```

3.3 Informative Priors per Metric

Metric	Base rate (Q1 2026 observed)	Informative prior	Notes
form_complete_rate	18%	Beta(19, 81)	Top-of-funnel; high volume, low sensitivity
tier_a_b_rate	42%	Beta(43, 57)	Mid-funnel; TS-10 routing decision
funded_loan_rate	5%	Beta(6, 94)	Bottom-funnel; low volume, high value
ctr	1.0%	Beta(11, 1089)	Per-ad-set CTR; high volume
cpl (lead)	\$185	Normal($\mu=185$, $\sigma=25$)	Continuous metric; uses Normal model
cpfl (funded loan)	\$3,800	Normal($\mu=3800$, $\sigma=600$)	Continuous; primary economics metric

3.4 Decision Threshold Calibration

The default `decision_threshold = 0.95` is appropriate for high-stakes decisions (model promotion, budget scaling). For lower-stakes creative rotation tests, set `decision_threshold = 0.90` to enable faster decisions. The threshold must be set in the `ABTestConfig` before test launch and **cannot** be changed mid-test (anti-p-hacking guardrail). Any threshold change is logged in the A/B test registry with sign-off.

3.5 Integration with Multi-Armed Bandit (Part 4)

For creative rotation (where many variants are tested simultaneously and the cost of a "loser" arm is just lost traffic, not a wrong model deploy), use the **multi-armed bandit** (Part 4) instead of a 2-arm Bayesian A/B test. The bandit continuously reallocates traffic to the best-performing arm. Use A/B tests (Part 3) for binary high-stakes decisions (model promotion, landing-page redesign, lead-magnet choice); use bandits (Part 4) for ongoing creative rotation across 3+ arms.

Part 4 — Multi-Armed Bandit Creative Rotation (Thompson Sampling)

4.1 Why a Bandit (Not a Fixed A/B Test) for Creative Rotation

V1 specified a fixed 9-test A/B calendar with each test running for ~5 weeks. For creative rotation (5+ ad hooks per persona, refreshed every 6–8 weeks per AC-09 V2), this is operationally infeasible:

- 20 personas × 6 hooks × 5-week test = 600 test-weeks per quarter. Not enough traffic.
- A fixed A/B test wastes traffic on underperforming arms for the full test duration.
- Creative fatigue (V2-9 in AC-09 V2) requires continuous rotation, not quarterly batch testing.

GODMODE uses **Thompson sampling** — a Bayesian multi-armed bandit — for creative rotation. Each "arm" is an ad creative (e.g., SA-001-PI-1, SA-001-PI-2, SA-001-PA-1, SA-001-PA-2, SA-001-PS-1, SA-001-PS-2). The bandit:

- Maintains a Beta(α , β) posterior for each arm's conversion rate.
- For each impression, samples one posterior draw per arm and serves the arm with the highest sample.
- Updates the served arm's posterior with the observed reward (1 = conversion, 0 = no conversion).
- Naturally balances exploration (high-uncertainty arms get sampled) and exploitation (high-mean arms dominate).

Thompson sampling is **asymptotically optimal** (sublinear regret) and **peek-safe** (no false-positive inflation from continuous monitoring).

4.2 Production Implementation

'''

Thompson Sampling Multi-Armed Bandit for DSCR Ad Creative Rotation

```

=====

Replaces V1's fixed quarterly A/B test calendar for creative rotation.
Each "arm" is an ad creative. Posterior is Beta-Binomial (conjugate).

Features:
- Per-arm Beta( $\alpha$ ,  $\beta$ ) posterior
- Per-impression sampling (true Thompson)
- Optional batch updates (per-day instead of per-impression – for high-volume)
- Posteriors persisted to warehouse daily (audit trail)
- Cold-start: new arms initialized to Beta(1, 1) until 100 impressions observed
- Auto-retire: arms below 1st-percentile posterior mean for 14 consecutive days retired
- Compliance: arm reward signal = "Tier_Routed_A_or_B" (NOT raw Form_Complete)
  per TS-10 Part 2A conversion-tracking contract
"""

from __future__ import annotations

import json
import logging
import math
import os
from dataclasses import dataclass, field, asdict
from datetime import datetime, timezone, timedelta
from typing import Any, Dict, Iterable, List, Optional, Tuple

import numpy as np
import pandas as pd
from scipy.stats import beta as beta_dist

logger = logging.getLogger("dscr.bandit")

# -----
# Thompson Sampler
# -----
class ThompsonSampler:
    """Multi-armed bandit for ad creative rotation using Thompson sampling.

    Each arm's reward posterior is Beta( $\alpha$ ,  $\beta$ ). Reward = 1 if the impression
    led to a Tier_Routed_A_or_B conversion within the attribution window
    (default 7 days), else 0.
    """

    def __init__(
        self,
        arms: List[str],
        prior_alpha: int = 1,
        prior_beta: int = 1,
        cold_start_impressions: int = 100,
        retirement_percentile: float = 0.01,
        retirement_window_days: int = 14,
        attribution_window_days: int = 7,
    ) -> None:
        """Initialize Thompson sampler.

    Args:
        arms: list of creative IDs (e.g., ['SA-001-PI-1', 'SA-001-PA-1', ...])
        prior_alpha: prior successes (default 1 = uniform prior Beta(1,1))
        prior_beta: prior failures (default 1 = uniform prior Beta(1,1))
        cold_start_impressions: minimum impressions before an arm is
            eligible for retirement (default 100)
        retirement_percentile: arms below this percentile of posterior mean
            for `retirement_window_days` consecutive days are auto-retired
        retirement_window_days: consecutive days below percentile to retire

```

```

        attribution_window_days: days after impression to count a conversion
"""
if not arms:
    raise ValueError("arms list cannot be empty")
self.arms: List[str] = list(arms)
self.prior_alpha = prior_alpha
self.prior_beta = prior_beta
self.alpha: Dict[str, int] = {arm: prior_alpha for arm in self.arms}
self.beta: Dict[str, int] = {arm: prior_beta for arm in self.arms}
self.impressions: Dict[str, int] = {arm: 0 for arm in self.arms}
self.conversions: Dict[str, int] = {arm: 0 for arm in self.arms}
self.retired: Dict[str, bool] = {arm: False for arm in self.arms}
self.cold_start_impressions = cold_start_impressions
self.retirement_percentile = retirement_percentile
self.retirement_window_days = retirement_window_days
self.attribution_window_days = attribution_window_days
# Per-arm rolling window of posterior means (for retirement logic)
self._posterior_history: Dict[str, List[Tuple[datetime, float]]] = {
    arm: [] for arm in self.arms
}

# -----
# Add / retire arms (for AC-09 V2 creative library refresh)
# -----
def add_arm(self, arm: str, prior_alpha: Optional[int] = None, prior_beta: Optional[int] = None) -> None:
    """Add a new arm. Defaults to cold-start prior Beta(1, 1)."""
    if arm in self.alpha:
        logger.warning("Arm %s already exists - no-op", arm)
        return
    self.arms.append(arm)
    self.alpha[arm] = prior_alpha if prior_alpha is not None else self.prior_alpha
    self.beta[arm] = prior_beta if prior_beta is not None else self.prior_beta
    self.impressions[arm] = 0
    self.conversions[arm] = 0
    self.retired[arm] = False
    self._posterior_history[arm] = []
    logger.info("Added arm %s (α=%d, β=%d)", arm, self.alpha[arm], self.beta[arm])

def retire_arm(self, arm: str, reason: str = "manual") -> None:
    """Manually retire an arm."""
    if arm not in self.alpha:
        raise KeyError(f"Arm {arm} not found")
    self.retired[arm] = True
    logger.info("Retired arm %s (reason: %s) - final α=%d, β=%d",
                arm, reason, self.alpha[arm], self.beta[arm])

def _active_arms(self) -> List[str]:
    return [arm for arm in self.arms if not self.retired[arm]]

# -----
# Core Thompson sampling
# -----
def select_arm(self) -> str:
    """Thompson sampling: select arm with highest posterior sample.

    For each active arm, draw one sample from Beta(α, β). Return the arm
    with the highest draw. This naturally balances exploration (high-
    uncertainty arms get sampled) and exploitation (high-mean arms dominate).
    """
    active = self._active_arms()
    if not active:
        raise RuntimeError("All arms retired - add a new arm or reset retired arms")
    samples = {
        arm: float(beta_dist.rvs(self.alpha[arm], self.beta[arm], random_state=None))
        for arm in active
    }

```

```

    }
    selected = max(samples, key=samples.get)
    return selected

def select_arms_batch(self, n: int) -> List[str]:
    """Select n arms (with replacement) for a batch of impressions.
    Useful for pre-computing allocations per ad set per day.
    """
    return [self.select_arm() for _ in range(n)]

def update(self, arm: str, reward: int, n_impressions: int = 1) -> None:
    """Update posterior after observing reward(s).

    Args:
        arm: creative ID
        reward: number of conversions observed
        n_impressions: number of impressions in this batch (default 1)
    """
    if arm not in self.alpha:
        raise KeyError(f"Arm {arm} not found")
    if reward < 0 or reward > n_impressions:
        raise ValueError(f"reward {reward} must be in [0, {n_impressions}]")
    self.alpha[arm] += reward
    self.beta[arm] += (n_impressions - reward)
    self.impressions[arm] += n_impressions
    self.conversions[arm] += reward

# -----
# Diagnostics
# -----
def get_arm_probabilities(self) -> Dict[str, Dict[str, float]]:
    """Current estimated conversion rate per arm (posterior mean) + 95% HDI."""
    out: Dict[str, Dict[str, float]] = {}
    for arm in self.arms:
        a, b = self.alpha[arm], self.beta[arm]
        mean = a / (a + b)
        # 95% credible interval
        lo = float(beta_dist.ppf(0.025, a, b))
        hi = float(beta_dist.ppf(0.975, a, b))
        out[arm] = {
            "posterior_mean": float(mean),
            "ci_95_lower": lo,
            "ci_95_upper": hi,
            "alpha": a,
            "beta": b,
            "impressions": self.impressions[arm],
            "conversions": self.conversions[arm],
            "retired": self.retired[arm],
        }
    return out

def get_exploration_rates(self) -> Dict[str, float]:
    """Current exploration rate per arm = (α + β) / sum across active arms.

    Arms with low total count have higher uncertainty → more exploration.
    """
    active = self._active_arms()
    total = sum(self.alpha[a] + self.beta[a] for a in active)
    if total == 0:
        return {arm: 1.0 / len(active) for arm in active}
    return {arm: (self.alpha[arm] + self.beta[arm]) / total for arm in active}

def get_traffic_allocation(self) -> Dict[str, float]:
    """Recommended traffic allocation per arm (Monte Carlo estimate).

```

```

    Samples 10,000 draws per arm, returns fraction of draws where each
    arm had the highest sample. This is the expected traffic share under
    Thompson sampling.
    """
    active = self._active_arms()
    if not active:
        return {}
    n_mc = 10_000
    draws = np.array([
        beta_dist.rvs(self.alpha[arm], self.beta[arm], size=n_mc)
        for arm in active
    ]) # shape (n_active_arms, n_mc)
    winners = np.argmax(draws, axis=0)
    allocation = {active[i]: float(np.mean(winners == i)) for i in range(len(active))}
    return allocation

# -----
# Auto-retirement (poor-performer kill switch)
# -----
def check_retirements(self, current_date: Optional[datetime] = None) -> List[str]:
    """Auto-retire arms below 1st-percentile posterior mean for 14 consecutive days.

    Returns list of arms retired in this check.
    """
    current_date = current_date or datetime.now(timezone.utc)
    probs = self.get_arm_probabilities()
    active = self._active_arms()
    if len(active) < 3:
        return [] # don't retire if only 1-2 arms left
    means = {arm: probs[arm]["posterior_mean"] for arm in active}
    # 1st percentile threshold
    threshold = float(np.percentile(list(means.values()), 100 * self.retirement_percentile))
    # Record today's posterior mean
    for arm in active:
        self._posterior_history[arm].append((current_date, means[arm]))

    # Check 14-day trailing window
    retired_today: List[str] = []
    cutoff = current_date - timedelta(days=self.retirement_window_days)
    for arm in active:
        if self.impressions[arm] < self.cold_start_impressions:
            continue # cold-start protection
        history = [(d, m) for d, m in self._posterior_history[arm] if d >= cutoff]
        if len(history) < self.retirement_window_days:
            continue # not enough history
        if all(m <= threshold for _, m in history):
            self.retire_arm(arm, reason=f"auto_retire_below_p{int(self.retirement_percentile*100)}_for_{self.retirement_window_days}days")
            retired_today.append(arm)
    return retired_today

# -----
# Persistence (for audit trail + crash recovery)
# -----
def to_dict(self) -> Dict[str, Any]:
    """Serialize sampler state for persistence to warehouse."""
    return {
        "arms": self.arms,
        "alpha": self.alpha,
        "beta": self.beta,
        "impressions": self.impressions,
        "conversions": self.conversions,
        "retired": self.retired,
        "prior_alpha": self.prior_alpha,
        "prior_beta": self.prior_beta,
        "cold_start_impressions": self.cold_start_impressions,
    }

```

```

        "retirement_percentile": self.retirement_percentile,
        "retirement_window_days": self.retirement_window_days,
        "attribution_window_days": self.attribution_window_days,
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
    }

    @classmethod
    def from_dict(cls, state: Dict[str, Any]) -> "ThompsonSampler":
        """Reconstruct sampler from persisted state."""
        sampler = cls(
            arms=state["arms"],
            prior_alpha=state["prior_alpha"],
            prior_beta=state["prior_beta"],
            cold_start_impressions=state["cold_start_impressions"],
            retirement_percentile=state["retirement_percentile"],
            retirement_window_days=state["retirement_window_days"],
            attribution_window_days=state["attribution_window_days"],
        )
        sampler.alpha = state["alpha"]
        sampler.beta = state["beta"]
        sampler.impressions = state["impressions"]
        sampler.conversions = state["conversions"]
        sampler.retired = state["retired"]
        return sampler

# -----
# Per-persona bandit manager (20 bandits, one per persona + edge case)
# -----
class PersonaBanditManager:
    """Manages 20 Thompson samplers, one per persona (SA-001..SA-012 + EG-001..EG-008).

    Each persona has its own creative library (AC-09 V2: 6 hooks per persona =
    PI-1, PI-2, PA-1, PA-2, PS-1, PS-2). The bandit rotates within persona.
    """

    PERSONA_CREATIVE_LIBRARY = {
        # SA-001 through SA-012
        **{f"SA-{i:03d}": [f"SA-{i:03d}-{cat}-{n}" for cat in ["PI", "PA", "PS"] for n in [1, 2]]
           for i in range(1, 13)},
        # EG-001 through EG-008
        **{f"EG-{i:03d}": [f"EG-{i:03d}-{cat}-{n}" for cat in ["PI", "PA", "PS"] for n in [1, 2]]
           for i in range(1, 9)},
    }

    def __init__(self) -> None:
        self.bandits: Dict[str, ThompsonSampler] = {
            persona: ThompsonSampler(arms=creatives)
            for persona, creatives in self.PERSONA_CREATIVE_LIBRARY.items()
        }

    def select_creative(self, persona: str) -> str:
        if persona not in self.bandits:
            raise KeyError(f"Unknown persona {persona}")
        return self.bandits[persona].select_arm()

    def update_creative(self, persona: str, creative: str, reward: int, n_impressions: int = 1) -> None:
        if persona not in self.bandits:
            raise KeyError(f"Unknown persona {persona}")
        self.bandits[persona].update(creative, reward, n_impressions)

    def daily_check_all(self) -> Dict[str, List[str]]:
        """Run auto-retirement check on all 20 bandits. Returns retired creatives per persona."""
        retired = {}
        for persona, bandit in self.bandits.items():

```

```

        retired[persona] = bandit.check_retirements()
    return retired

def export_state(self) -> Dict[str, Any]:
    return {persona: bandit.to_dict() for persona, bandit in self.bandits.items()}

def snapshot_to_warehouse(self, warehouse_conn) -> None:
    """Persist bandit state to warehouse for audit + crash recovery."""
    rows = []
    ts = datetime.now(timezone.utc).isoformat()
    for persona, bandit in self.bandits.items():
        probs = bandit.get_arm_probabilities()
        alloc = bandit.get_traffic_allocation()
        for arm, stats in probs.items():
            rows.append({
                "snapshot_ts_utc": ts,
                "persona": persona,
                "creative_id": arm,
                "alpha": stats["alpha"],
                "beta": stats["beta"],
                "posterior_mean": stats["posterior_mean"],
                "ci_95_lower": stats["ci_95_lower"],
                "ci_95_upper": stats["ci_95_upper"],
                "impressions": stats["impressions"],
                "conversions": stats["conversions"],
                "retired": stats["retired"],
                "traffic_allocation": alloc.get(arm, 0.0),
            })
    df = pd.DataFrame(rows)
    df.to_sql("fact_bandit_snapshots", warehouse_conn,
              schema="dscr_prod", if_exists="append", index=False)
    logger.info("Persisted bandit snapshot - %d rows", len(df))

# -----
# Example: daily bandit update job (Dagster op)
# -----
def daily_bandit_update_job(warehouse_conn, persona_bandit_manager: PersonaBanditManager) -> None:
    """Daily Dagster job: pull yesterday's impressions + conversions per creative,
    update bandits, check retirements, snapshot to warehouse.

    Reward signal = Tier_Routed_A_or_B (per TS-10 Part 2A conversion-tracking contract).
    NOT raw Form_Complete (which would teach the bandit to attract unqualified leads).
    """
    yesterday = (datetime.now(timezone.utc) - timedelta(days=1)).date()
    query = """
        SELECT
            a.persona_targeted AS persona,
            a.ad_creative_id AS creative,
            COUNT(*) AS impressions,
            COUNT(l.lead_id) AS form_completes,
            COUNT(CASE WHEN l.tier_routed IN ('A', 'B') THEN 1 END) AS tier_a_b_conversions
        FROM dscr_prod.fact_ad_spend a
        LEFT JOIN dscr_prod.fact_leads l
            ON l.ad_creative_id = a.ad_creative_id
            AND l.created_date::date = a.spend_date::date
            AND l.created_date <= a.spend_date + INTERVAL '7 days'
        WHERE a.spend_date::date = %(yesterday)s
        GROUP BY 1, 2
    """
    df = pd.read_sql(query, con=warehouse_conn, params={"yesterday": yesterday})
    logger.info("Pulled %d creative rows for %s", len(df), yesterday)

    for _, row in df.iterrows():
        persona = row["persona"]

```

```

creative = row["creative"]
if persona not in persona_bandit_manager.bandits:
    logger.warning("Unknown persona %s - skipping", persona)
    continue
# Reward = Tier_Routed_A_or_B conversions (NOT raw Form_Completes)
reward = int(row["tier_a_b_conversions"])
n_impr = int(row["impressions"])
persona_bandit_manager.update_creative(persona, creative, reward, n_impr)

# Check retirements
retired = persona_bandit_manager.daily_check_all()
for persona, arms in retired.items():
    for arm in arms:
        logger.warning("Auto-retired creative %s on persona %s", arm, persona)

# Snapshot
persona_bandit_manager.snapshot_to_warehouse(warehouse_conn)

if __name__ == "__main__":
    # Smoke test
    sampler = ThompsonSampler(arms=["SA-001-PI-1", "SA-001-PA-1", "SA-001-PS-1"])
    # Simulate 1,000 impressions
    true_rates = {"SA-001-PI-1": 0.05, "SA-001-PA-1": 0.08, "SA-001-PS-1": 0.04}
    for _ in range(1000):
        arm = sampler.select_arm()
        reward = 1 if np.random.random() < true_rates[arm] else 0
        sampler.update(arm, reward)
    print("Posterior probabilities:")
    print(json.dumps(sampler.get_arm_probabilities(), indent=2))
    print("\nTraffic allocation:")
    print(json.dumps(sampler.get_traffic_allocation(), indent=2))

```

4.3 Bandit vs A/B Test Decision Matrix

Use case	Tool	Rationale
2-arm model promotion (v_current vs v_next)	Part 3 Bayesian A/B test	High-stakes; need precise effect size + ROPE; rollback capability
3+ creative variants within a persona	Part 4 bandit	Continuous rotation; auto-retire losers; minimize regret
Landing page redesign (2 variants)	Part 3 Bayesian A/B test	High implementation cost; need definitive winner
Lead-magnet choice (3 variants)	Part 4 bandit	Cheap to rotate; learn over time
Channel budget allocation (Meta vs Google vs LinkedIn)	Part 3 Bayesian A/B test	High \$ stakes; need HDI on effect size
Hook category (PI vs PA vs PS) per persona	Part 4 bandit	6 arms per persona; continuous learning
Geo-holdout (treatment DMAs vs holdout DMAs)	Part 5 Causal Impact	Not an A/B test — needs causal inference, not bandit

4.4 Compliance Notes

- **Reward signal is** `Tier_Routed_A_or_B` — not raw `Form_Complete`. This is the TS-10 Part 2A contract: optimizing on raw `Form_Complete` teaches the bandit to attract NP-011 (zero-reserves) and NP-008 (unpermitted-ADU) decline cohorts.
- **No protected-class features in arm selection.** Arms are creative IDs, not audience segments. Audience targeting is set at the ad-set level (TS-10 Part 2A).
- **Auto-retirement log is the audit trail.** Every retirement (manual or auto) is persisted to `fact_bandit_snapshots` with timestamp + reason. Reviewed at QIC day 10.

Part 5 — Causal Impact Analysis (Geo-Holdout)

5.1 Purpose

Geo-holdout testing isolates the **incremental causal lift** of paid advertising spend, distinct from organic demand, seasonality, and macroeconomic drift. Unlike a user-level A/B test (Part 3) or a multi-armed bandit (Part 4), a geo-holdout randomizes at the **DMA (Designated Market Area)** level and uses synthetic control to construct a counterfactual.

The CausalImpact library (Brodersen et al., 2015) fits a Bayesian structural time-series model: a synthetic control series is constructed from a weighted combination of holdout DMAs, then the post-intervention gap between treatment DMAs and the synthetic counterfactual is the **incremental lift** attributable to ad spend.

Decision rule (hard gate): if incremental ROAS $< 1.5\times$, the channel is paused pending QIC review. This is the single source of truth for ad-spend efficiency — paid-platform ROAS is contaminated by attribution windows and view-through credit.

5.2 Data Contract

- **Treatment DMAs:** DMAs where paid spend is active.
- **Holdout DMAs:** DMAs where paid spend is **zero** for the entire test window (matched on population + historical lead volume).
- **Granularity:** weekly aggregates per DMA.
- **Outcome metric:** count of `Tier_A_or_B_qualified_leads` (NOT raw leads — same compliance gate as Parts 2 and 4).
- **Pre-period:** ≥ 12 weeks before intervention to fit the synthetic control.
- **Post-period:** ≥ 8 weeks after intervention to estimate lift.

5.3 Production Code

11 11 11

```

Geo-holdout causal impact analyzer for paid-channel incremental ROAS.

Decision gate: if incremental ROAS < 1.5x, channel is paused pending QIC review.
"""
from __future__ import annotations

import logging
from dataclasses import dataclass
from datetime import date, datetime, timedelta
from typing import Any, Optional

import numpy as np
import pandas as pd

try:
    import causalimpact as ci
except ImportError as exc:  # pragma: no cover
    raise ImportError(
        "Install causalimpact: pip install causalimpact"
    ) from exc

from sqlalchemy import create_engine, text
from sqlalchemy.engine import Engine

logger = logging.getLogger(__name__)

@dataclass(frozen=True)
class CausalImpactResult:
    """Container for a single geo-holdout causal-impact run."""

    channel: str
    pre_period_start: date
    pre_period_end: date
    post_period_start: date
    post_period_end: date
    incremental_leads: float
    incremental_leads_ci_lower: float
    incremental_leads_ci_upper: float
    cumulative_spend: float
    incremental_roas: float
    incremental_roas_ci_lower: float
    incremental_roas_ci_upper: float
    p_value: float
    decision: str  # "KEEP" | "PAUSE" | "INCONCLUSIVE"

    def to_dict(self) -> dict[str, Any]:
        return {
            "channel": self.channel,
            "pre_period_start": self.pre_period_start.isoformat(),
            "pre_period_end": self.pre_period_end.isoformat(),
            "post_period_start": self.post_period_start.isoformat(),
            "post_period_end": self.post_period_end.isoformat(),
            "incremental_leads": round(self.incremental_leads, 2),
            "incremental_leads_ci_lower": round(self.incremental_leads_ci_lower, 2),
            "incremental_leads_ci_upper": round(self.incremental_leads_ci_upper, 2),
            "cumulative_spend": round(self.cumulative_spend, 2),
            "incremental_roas": round(self.incremental_roas, 4),
            "incremental_roas_ci_lower": round(self.incremental_roas_ci_lower, 4),
            "incremental_roas_ci_upper": round(self.incremental_roas_ci_upper, 4),
            "p_value": round(self.p_value, 4),
            "decision": self.decision,
        }

# Decision thresholds (tunable via config; defaults are TS-10 contract)

```

```

ROAS_PAUSE_THRESHOLD: float = 1.5
ROAS_INCONCLUSIVE_BAND: float = 0.2  # ±0.2 around threshold = inconclusive
SIGNIFICANCE_ALPHA: float = 0.05

class GeoHoldoutAnalyzer:
    """Run CausalImpact on treatment vs. holdout DMAs for a single channel."""

    def __init__(
        self,
        warehouse_uri: str,
        treatment_dmas: list[str],
        holdout_dmas: list[str],
        channel: str,
        roas_pause_threshold: float = ROAS_PAUSE_THRESHOLD,
    ) -> None:
        if not treatment_dmas or not holdout_dmas:
            raise ValueError("Both treatment and holdout DMA lists must be non-empty.")
        if len(holdout_dmas) < 3:
            raise ValueError(
                "Need >=3 holdout DMAs for stable synthetic control "
                f"(got {len(holdout_dmas)})."
            )
        if channel in treatment_dmas:
            raise ValueError("Channel name cannot collide with DMA id.")

        self.engine: Engine = create_engine(warehouse_uri)
        self.treatment_dmas = treatment_dmas
        self.holdout_dmas = holdout_dmas
        self.channel = channel
        self.roas_pause_threshold = roas_pause_threshold

    # ----- #
    # Data prep
    # ----- #
    def prepare_data(
        self,
        start_date: date,
        end_date: date,
    ) -> pd.DataFrame:
        """
        Pull weekly Tier A-or-B qualified leads per DMA for treatment + holdout.

        Returns a DataFrame indexed by week with columns:
        - week_start (datetime, tz-naive UTC)
        - treatment (sum across treatment DMAs)
        - h1, h2, ... hN (one column per holdout DMA)
        """
        all_dmas = self.treatment_dmas + self.holdout_dmas
        dma_sql_literal = ",".join(f'"{d}"' for d in all_dmas)

        sql = text(
            f"""
            WITH weekly AS (
                SELECT
                    DATE_TRUNC('week', l.created_at)::date AS week_start,
                    l.dma_id,
                    COUNT(DISTINCT l.lead_id) FILTER (
                        WHERE lc.tier_routed IN ('A', 'B')
                    ) AS tier_ab_leads
                FROM fact_leads l
                LEFT JOIN dim_lead_classification lc
                    ON lc.lead_id = l.lead_id
                WHERE l.dma_id IN ({dma_sql_literal})
                    AND l.created_at >= :start_date
            """
        )

```

```

        AND l.created_at <= :end_date
        AND l.is_test_lead = FALSE
    GROUP BY 1, 2
)
SELECT
    w.week_start,
    w.dma_id,
    COALESCE(w.tier_ab_leads, 0) AS tier_ab_leads
FROM weekly w
ORDER BY w.week_start, w.dma_id;
"""
)

with self.engine.connect() as conn:
    raw = pd.read_sql(
        sql,
        conn,
        params={"start_date": start_date, "end_date": end_date},
    )

    if raw.empty:
        raise ValueError(
            f"No leads found in window {start_date} -> {end_date} "
            f"for DMAs {all_dmas}."
        )

    pivot = raw.pivot_table(
        index="week_start",
        columns="dma_id",
        values="tier_ab_leads",
        fill_value=0,
        aggfunc="sum",
    ).sort_index()

    # Combine treatment DMAs into a single column
    missing_treatment = set(self.treatment_dmas) - set(pivot.columns)
    if missing_treatment:
        raise ValueError(
            f"Treatment DMAs missing from data: {missing_treatment}"
        )
    treatment_series = pivot[self.treatment_dmas].sum(axis=1)
    treatment_series.name = "treatment"

    holdout_df = pivot[self.holdout_dmas].copy()
    # Rename holdout columns to h1, h2, ...
    holdout_df.columns = [f"h{i+1}" for i in range(len(holdout_df.columns))]

    out = pd.concat([treatment_series, holdout_df], axis=1)
    out.index = pd.to_datetime(out.index)
    return out

# ----- #
# Spend lookup
# ----- #
def _fetch_spend(
    self,
    start_date: date,
    end_date: date,
) -> float:
    """Cumulative paid spend on this channel in treatment DMAs only."""
    dma_sql_literal = ",".join(f"'{d}'" for d in self.treatment_dmas)
    sql = text(
        f"""
        SELECT COALESCE(SUM(spend_usd), 0) AS total_spend
        FROM fact_ad_spend

```

```

        WHERE channel = :channel
            AND dma_id IN ({dma_sql_literal})
            AND spend_date >= :start_date
            AND spend_date < :end_date;
        """
    )
    with self.engine.connect() as conn:
        row = conn.execute(
            sql,
            params={
                "channel": self.channel,
                "start_date": start_date,
                "end_date": end_date,
            },
        ).fetchone()
    if row is None or row[0] is None:
        return 0.0
    return float(row[0])

# ----- #
# Core analysis
# ----- #
def run_analysis(
    self,
    pre_period: tuple[date, date],
    post_period: tuple[date, date],
) -> CausalImpactResult:
    """
    Run CausalImpact on (treatment, holdout controls).

    Parameters
    -----
    pre_period : (start, end) of pre-intervention window (>= 12 weeks)
    post_period : (start, end) of post-intervention window (>= 8 weeks)
    """
    pre_start, pre_end = pre_period
    post_start, post_end = post_period
    if post_start <= pre_end:
        raise ValueError("post_period must start strictly after pre_period ends.")
    if (pre_end - pre_start).days < 84:
        raise ValueError("Pre-period must be >= 12 weeks (84 days).")
    if (post_end - post_start).days < 56:
        raise ValueError("Post-period must be >= 8 weeks (56 days).")

    full_start = pre_start
    full_end = post_end
    df = self.prepare_data(full_start, full_end)

    # CausalImpact expects integer-indexed pre/post window pairs
    pre_mask = (df.index >= pd.Timestamp(pre_start)) & (
        df.index <= pd.Timestamp(pre_end)
    )
    post_mask = (df.index >= pd.Timestamp(post_start)) & (
        df.index <= pd.Timestamp(post_end)
    )
    if pre_mask.sum() < 8 or post_mask.sum() < 4:
        raise ValueError(
            f"Insufficient observations: pre={pre_mask.sum()}, "
            f"post={post_mask.sum()}."
        )

    pre_idx = [int(np.where(pre_mask.values)[0][0]),
               int(np.where(pre_mask.values)[0][-1])]
    post_idx = [int(np.where(post_mask.values)[0][0]),
               int(np.where(post_mask.values)[0][-1])]

```

```

pre_period_tuple = (pre_idx[0], pre_idx[1])
post_period_tuple = (post_idx[0], post_idx[1])

logger.info(
    "Running CausalImpact for channel=%s pre=%s..%s post=%s..%s",
    self.channel, pre_start, pre_end, post_start, post_end,
)

try:
    impact = ci.CausalImpact(
        data=df,
        pre_period=pre_period_tuple, # type: ignore[arg-type]
        post_period=post_period_tuple, # type: ignore[arg-type]
        niter=2000,
        standardize=True,
    )
except Exception as exc:
    logger.exception("CausalImpact failed for channel=%s", self.channel)
    raise RuntimeError(
        f"CausalImpact failed for channel {self.channel}: {exc}"
    ) from exc

summary = impact.summary_data # dict-like with keys
# CausalImpact summary keys (v0.x API):
# 'average', 'cumulative', 'p', 'abs_effect', 'abs_effect_lower',
# 'abs_effect_upper', 'rel_effect', 'rel_effect_lower', 'rel_effect_upper'
cumulative_abs = float(summary["cumulative"]["abs_effect"])
cumulative_lower = float(summary["cumulative"]["abs_effect_lower"])
cumulative_upper = float(summary["cumulative"]["abs_effect_upper"])
p_value = float(summary["p"])

cumulative_spend = self._fetch_spend(post_start, post_end)
if cumulative_spend <= 0:
    logger.warning(
        "Zero spend recorded for channel=%s between %s and %s - "
        "ROAS will be infinite; marking INCONCLUSIVE.",
        self.channel, post_start, post_end,
    )
    return CausalImpactResult(
        channel=self.channel,
        pre_period_start=pre_start,
        pre_period_end=pre_end,
        post_period_start=post_start,
        post_period_end=post_end,
        incremental_leads=cumulative_abs,
        incremental_leads_ci_lower=cumulative_lower,
        incremental_leads_ci_upper=cumulative_upper,
        cumulative_spend=0.0,
        incremental_roas=float("inf"),
        incremental_roas_ci_lower=float("inf"),
        incremental_roas_ci_upper=float("inf"),
        p_value=p_value,
        decision="INCONCLUSIVE",
    )

# ROAS = incremental leads / spend (leads per dollar; multiply by
# value-per-lead for $-ROAS - here we use leads/$1 as a proxy unit)
inc_roas = cumulative_abs / cumulative_spend
inc_roas_lower = cumulative_lower / cumulative_spend
inc_roas_upper = cumulative_upper / cumulative_spend

decision = self._decide(inc_roas, inc_roas_lower, inc_roas_upper, p_value)

return CausalImpactResult(
    channel=self.channel,

```

```

        pre_period_start=pre_start,
        pre_period_end=pre_end,
        post_period_start=post_start,
        post_period_end=post_end,
        incremental_leads=cumulative_abs,
        incremental_leads_ci_lower=cumulative_lower,
        incremental_leads_ci_upper=cumulative_upper,
        cumulative_spend=cumulative_spend,
        incremental_roas=inc_roas,
        incremental_roas_ci_lower=inc_roas_lower,
        incremental_roas_ci_upper=inc_roas_upper,
        p_value=p_value,
        decision=decision,
    )

# ----- #
# Decision rule
# ----- #
def _decide(
    self,
    roas: float,
    roas_lower: float,
    roas_upper: float,
    p_value: float,
) -> str:
    """
    Hard gate: pause channel if incremental ROAS < threshold AND
    the upper CI bound is also below (threshold + inconclusive band)
    AND p_value < alpha.
    """
    if not np.isfinite(roas):
        return "INCONCLUSIVE"
    if p_value >= SIGNIFICANCE_ALPHA:
        # Effect not statistically distinguishable from zero
        return "INCONCLUSIVE"
    if roas < self.roas_pause_threshold and roas_upper < (
        self.roas_pause_threshold + ROAS_INCONCLUSIVE_BAND
    ):
        return "PAUSE"
    if roas_lower < self.roas_pause_threshold <= roas_upper:
        # CI straddles the threshold – inconclusive
        return "INCONCLUSIVE"
    return "KEEP"

# ----- #
# Persistence
# ----- #
def persist(self, result: CausalImpactResult) -> None:
    """Write the decision row to fact_geo_holdout_decisions."""
    sql = text(
        """
        INSERT INTO fact_geo_holdout_decisions (
            channel, pre_period_start, pre_period_end,
            post_period_start, post_period_end,
            incremental_leads, incremental_leads_ci_lower,
            incremental_leads_ci_upper, cumulative_spend,
            incremental_roas, incremental_roas_ci_lower,
            incremental_roas_ci_upper, p_value, decision,
            run_at
        ) VALUES (
            :channel, :ps, :pe, :pos_s, :pos_e,
            :il, :il_l, :il_u, :spend,
            :roas, :roas_l, :roas_u, :p, :decision,
            NOW()
        );
    """
    )

```

```

    """
)
d = result.to_dict()
with self.engine.begin() as conn:
    conn.execute(
        sql,
        params={
            "channel": d["channel"],
            "ps": d["pre_period_start"],
            "pe": d["pre_period_end"],
            "pos_s": d["post_period_start"],
            "pos_e": d["post_period_end"],
            "il": d["incremental_leads"],
            "il_l": d["incremental_leads_ci_lower"],
            "il_u": d["incremental_leads_ci_upper"],
            "spend": d["cumulative_spend"],
            "roas": d["incremental_roas"],
            "roas_l": d["incremental_roas_ci_lower"],
            "roas_u": d["incremental_roas_ci_upper"],
            "p": d["p_value"],
            "decision": d["decision"],
        },
    )
logger.info("Persisted CausalImpact decision: %s", d)

```

5.4 Interpretation Guide

Outcome	What it means	Action
decision = KEEP and $p < 0.05$	Channel produces statistically significant incremental leads at $ROAS \geq 1.5\times$	Maintain or scale budget (QIC day 10 vote)
decision = PAUSE and $p < 0.05$	Channel's incremental ROAS is below the $1.5\times$ threshold and the upper CI bound does not cross into the inconclusive band	Pause channel; re-allocate budget to next-best channel per Part 4 bandit priors
decision = INCONCLUSIVE	Either $p \geq 0.05$ (no detectable effect) OR the CI straddles the threshold	Extend post-period by 4 weeks; re-run before QIC day 10
incremental_leads < 0 (CI excludes 0)	Channel is cannibalizing organic demand (negative lift)	Pause immediately; investigate attribution leakage

5.4.1 Common pitfalls

- Holdout contamination.** If a holdout DMA receives any paid spend during the post-period (even \$1), the synthetic control is corrupted. Run a daily monitor on holdout DMAs: `SELECT SUM(spend_usd) FROM fact_ad_spend WHERE dma_id IN (holdout) AND spend_date >= post_start`. Trigger alert if > 0 .
- DMA carryover.** Users in treatment DMAs may search / convert via organic in holdout DMAs (cross-border shopping). Mitigate by choosing non-adjacent holdouts.
- Seasonal confounders.** If a treatment DMA hosts a one-off event (e.g., major real-estate expo) during the post-period, the synthetic control cannot reproduce it. Add an event-flag column as an exogenous regressor via `CausalImpact(..., covariates=...)`.

4. **Underpowered pre-period.** <12 weeks of pre-period data leads to unstable synthetic control weights. Hard-fail the run.

5.5 Orchestrator (QIC day 5 entry point)

```
"""Run geo-holdout analysis for all active channels on QIC day 5."""
import logging
from datetime import date, timedelta

from analytics.causal_impact import GeoHoldoutAnalyzer

logging.basicConfig(level=logging.INFO)
log = logging.getLogger("qic.geo_holdout")

CHANNEL_DMA_MAP = {
    "meta_paid": {"treatment": ["DMA_501", "DMA_504", "DMA_511"],
                  "holdout": ["DMA_524", "DMA_539", "DMA_567", "DMA_582"]},
    "google_paid": {"treatment": ["DMA_602", "DMA_613", "DMA_618"],
                    "holdout": ["DMA_622", "DMA_640", "DMA_651", "DMA_662"]},
    "linkedin_paid": {"treatment": ["DMA_701", "DMA_707"],
                     "holdout": ["DMA_715", "DMA_725", "DMA_740"]},
}

def run_quarterly_geo_holdouts(run_date: date) -> None:
    post_end = run_date
    post_start = post_end - timedelta(days=56) # 8 weeks post
    pre_end = post_start - timedelta(days=1)
    pre_start = pre_end - timedelta(days=84) # 12 weeks pre

    decisions = []
    for channel, dmas in CHANNEL_DMA_MAP.items():
        try:
            analyzer = GeoHoldoutAnalyzer(
                warehouse_uri="postgres://warehouse_user@wh-prod/analytics",
                treatment_dmas=dmas["treatment"],
                holdout_dmas=dmas["holdout"],
                channel=channel,
            )
            result = analyzer.run_analysis(
                pre_period=(pre_start, pre_end),
                post_period=(post_start, post_end),
            )
            analyzer.persist(result)
            decisions.append(result)
        except Exception:
            log.exception("Failed channel=%s", channel)

    paused = [d for d in decisions if d.decision == "PAUSE"]
    log.info(
        "Geo-holdout complete. %d/%d channels marked PAUSE: %s",
        len(paused), len(decisions),
        [d.channel for d in paused],
    )
```

5.6 Part 5 Compliance Notes

- The outcome metric is Tier_A_or_B-qualified leads, not raw leads. This prevents the holdout from being credited with NP-011 (zero-reserves) and NP-008 (unpermitted-ADU) leads that

would route to Decline and never fund — preserving the TS-10 Part 2A contract.

- No protected-class features enter the synthetic-control covariates. Holdout DMA selection is based on historical lead volume + population, never on race/ethnicity/ZIP-code-as-proxy.
- All decisions are persisted with `run_at` timestamp for fair-lensing audit (Part 12).

Part 6 — Cohort Retention Curves (Funded-Loan Repeat Borrowing)

6.1 Purpose

A funded-loan cohort is the set of borrowers whose loans funded in a given calendar month. **Cohort retention** measures the fraction of that cohort that takes out a second (or third, Nth) loan within 6, 12, 18, and 24 months of the original funding date.

This metric drives two downstream decisions:

1. **FDI weighting.** Personas with high repeat-borrowing retention receive a higher FDI weighting (Part 2 ML scoring), because their lifetime value (Part 8 LTV) is materially higher.
2. **Persona-library edits.** A persona whose retention curve flattens at <10% by month 12 is flagged for QIC review — either the persona definition has drifted, or the LO handoff is failing.

6.2 Cohort Definition

- **Cohort key:** `funded_month = DATE_TRUNC('month', first_funded_loan_date)`.
- **Persona dimension:** the persona assigned to the lead at first funded loan (immutable — see Part 2).
- **Retention event:** a subsequent loan funding by the same `borrower_id` where `loan_number > 1`.
- **Retention windows:** months 6, 12, 18, 24 from `first_funded_loan_date`.
- **Right-censoring:** cohorts whose first-funded date is < 24 months before run date are censored at the available window.

6.3 Production Code

```
"""
Cohort retention analyzer for funded-loan repeat borrowing.

Outputs:
- Long-format retention table per (cohort_month, persona, window)
- Wide-format pivot for visualization
- Matplotlib retention curves by persona
"""
```

```

from __future__ import annotations

import io
import logging
from dataclasses import dataclass
from datetime import date, datetime
from typing import Optional

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sqlalchemy import create_engine, text
from sqlalchemy.engine import Engine

logger = logging.getLogger(__name__)

RETENTION_WINDOWS_MONTHS: tuple[int, ...] = (6, 12, 18, 24)

@dataclass(frozen=True)
class RetentionResult:
    cohort_month: date
    persona_id: str
    cohort_size: int
    window_months: int
    retained_count: int
    retention_rate: float
    is_censored: bool

class CohortRetentionAnalyzer:
    """Compute funded-loan cohort retention curves by persona."""

    def __init__(self, warehouse_uri: str) -> None:
        self.engine: Engine = create_engine(warehouse_uri)

    # ----- #
    # SQL: pull first-funded + repeat-funded events per borrower
    # ----- #
    def _fetch_cohort_data(self, as_of: date) -> pd.DataFrame:
        sql = text(
            """
            WITH first_funded AS (
                SELECT
                    b.borrower_id,
                    b.persona_id_at_first_funded,
                    MIN(fl.funded_date) AS first_funded_date
                FROM fact_loans fl
                JOIN dim_borrowers b ON b.borrower_id = fl.borrower_id
                WHERE fl.loan_status = 'FUNDED'
                GROUP BY 1, 2
            ),
            repeat_funded AS (
                SELECT
                    b.borrower_id,
                    fl.funded_date AS repeat_funded_date,
                    ROW_NUMBER() OVER (
                        PARTITION BY b.borrower_id
                        ORDER BY fl.funded_date
                    ) AS loan_ordinal
                FROM fact_loans fl
                JOIN dim_borrowers b ON b.borrower_id = fl.borrower_id
                WHERE fl.loan_status = 'FUNDED'
            )
            SELECT

```

```

        ff.borrower_id,
        ff.persona_id_at_first_funded AS persona_id,
        ff.first_funded_date,
        rf.repeat_funded_date,
        rf.loan_ordinal,
        DATE_TRUNC('month', ff.first_funded_date)::date AS cohort_month
    FROM first_funded ff
    LEFT JOIN repeat_funded rf
        ON rf.borrower_id = ff.borrower_id
        AND rf.loan_ordinal > 1
    WHERE ff.first_funded_date <= :as_of
    ORDER BY ff.first_funded_date;
    """
)
with self.engine.connect() as conn:
    df = pd.read_sql(sql, conn, params={"as_of": as_of})
    return df

# ----- #
# Compute retention
# ----- #
def compute_retention(
    self,
    as_of: date,
    windows_months: tuple[int, ...] = RETENTION_WINDOWS_MONTHS,
) -> pd.DataFrame:
    """
    Returns a long-format DataFrame:
    cohort_month | persona_id | window_months | cohort_size |
    retained_count | retention_rate | is_censored
    """
    raw = self._fetch_cohort_data(as_of)
    if raw.empty:
        return pd.DataFrame(
            columns=[
                "cohort_month", "persona_id", "window_months",
                "cohort_size", "retained_count", "retention_rate",
                "is_censored",
            ]
        )

    as_of_ts = pd.Timestamp(as_of)
    raw["first_funded_date"] = pd.to_datetime(raw["first_funded_date"])
    raw["repeat_funded_date"] = pd.to_datetime(raw["repeat_funded_date"])

    rows: list[dict] = []
    # Group by (cohort_month, persona) -> borrowers
    for (cohort_month, persona_id), grp in raw.groupby(
        ["cohort_month", "persona_id"]
    ):
        cohort_size = grp["borrower_id"].nunique()
        first_dates = grp.groupby("borrower_id")["first_funded_date"].first()
        # earliest repeat per borrower
        repeat_first = (
            grp.dropna(subset=["repeat_funded_date"])
            .groupby("borrower_id")["repeat_funded_date"]
            .min()
        )

        for w in windows_months:
            window_end = first_dates + pd.DateOffset(months=w)
            is_censored = bool((window_end > as_of_ts).any())
            # A borrower is "retained" at window w if they have a repeat
            # funding on or before first_funded + w months.
            retained = 0

```

```

        for bid, fd in first_dates.items():
            if bid not in repeat_first.index:
                continue
            rd = repeat_first[bid]
            if pd.isna(rd):
                continue
            if rd <= fd + pd.DateOffset(months=w):
                retained += 1
        rate = retained / cohort_size if cohort_size else 0.0
        rows.append(
            {
                "cohort_month": pd.Timestamp(cohort_month).date(),
                "persona_id": persona_id,
                "window_months": w,
                "cohort_size": int(cohort_size),
                "retained_count": int(retained),
                "retention_rate": float(rate),
                "is_censored": is_censored,
            }
        )

    return pd.DataFrame(rows)

# ----- #
# Aggregation: persona-level mean retention across recent cohorts
# ----- #
def persona_retention_summary(
    self,
    as_of: date,
    lookback_months: int = 18,
) -> pd.DataFrame:
    """
    Returns one row per (persona_id, window_months) averaged across the
    last `lookback_months` of complete (non-censored) cohorts.
    """
    long = self.compute_retention(as_of)
    if long.empty:
        return long
    cutoff = pd.Timestamp(as_of) - pd.DateOffset(months=lookback_months)
    long["cohort_month"] = pd.to_datetime(long["cohort_month"])
    recent = long[long["cohort_month"] >= cutoff]
    summary = (
        recent.groupby(["persona_id", "window_months"])
        .agg(
            mean_retention_rate=("retention_rate", "mean"),
            median_retention_rate=("retention_rate", "median"),
            cohorts_observed=("cohort_month", "nunique"),
            total_cohort_size=("cohort_size", "sum"),
        )
        .reset_index()
    )
    return summary

# ----- #
# Visualization
# ----- #
def plot_retention_curves(
    self,
    summary: pd.DataFrame,
    output_path: str,
    title: str = "Funded-Loan Cohort Retention by Persona",
) -> str:
    """
    Render retention curves (one line per persona) and save to PNG.
    """

```

```

fig, ax = plt.subplots(figsize=(11, 6.5), dpi=140)
personas = summary["persona_id"].unique()
cmap = plt.cm.tab20(np.linspace(0, 1, len(personas)))

for color, persona in zip(cmap, personas):
    sub = (
        summary[summary["persona_id"] == persona]
        .sort_values("window_months")
    )
    ax.plot(
        sub["window_months"],
        sub["mean_retention_rate"] * 100,
        marker="o",
        linewidth=1.8,
        color=color,
        label=persona,
    )

ax.set_xlabel("Months since first funded loan")
ax.set_ylabel("Repeat-borrowing retention rate (%)")
ax.set_title(title)
ax.set_xticks(RETENTION_WINDOWS_MONTHS)
ax.grid(True, alpha=0.3)
ax.legend(
    loc="upper right",
    fontsize=8,
    ncol=2,
    framealpha=0.85,
    title="Persona",
)
ax.set_ylim(bottom=0)
fig.tight_layout()
fig.savefig(output_path, format="png")
plt.close(fig)
logger.info("Wrote retention chart -> %s", output_path)
return output_path

# ----- #
# FDI weighting helper
# ----- #
def fdi_weight_adjustment(
    self,
    summary: pd.DataFrame,
    baseline_rate: float = 0.20,
    max_multiplier: float = 1.5,
    min_multiplier: float = 0.7,
) -> pd.DataFrame:
    """
    Compute a per-persona FDI weighting multiplier from 24-month retention.

    Multiplier = clip(retention_24m / baseline_rate, min, max).
    Personas with >1.5x baseline retention get a +50% FDI boost;
    personas with <70% baseline retention get a -30% penalty.
    """
    m24 = summary[summary["window_months"] == 24].copy()
    if m24.empty:
        # Fall back to 18m if 24m not available
        m24 = summary[summary["window_months"] == 18].copy()
    if m24.empty:
        raise ValueError("No 18m or 24m retention data available.")

    m24["fdi_multiplier"] = (
        m24["mean_retention_rate"] / baseline_rate
    ).clip(lower=min_multiplier, upper=max_multiplier)
    return m24[

```

```
[ "persona_id", "mean_retention_rate", "fdi_multiplier" ]
].rename(columns={"mean_retention_rate": "retention_24m"})
```

6.4 Orchestration: monthly refresh

```
"""Monthly cohort-retention refresh — runs first business day of each month."""
from datetime import date, timedelta

from analytics.cohort_retention import CohortRetentionAnalyzer

def monthly_retention_run(run_date: date) -> None:
    analyzer = CohortRetentionAnalyzer(
        warehouse_uri="postgresql://warehouse_user@wh-prod/analytics"
    )
    summary = analyzer.persona_retention_summary(
        as_of=run_date, lookback_months=18
    )
    summary.to_csv(f"/artifacts/retention/retention_{run_date.isoformat()}.csv",
                  index=False)

    chart_path = (
        f"/artifacts/retention/retention_chart_{run_date.isoformat()}.png"
    )
    analyzer.plot_retention_curves(summary, chart_path)

    fdi_adj = analyzer.fdi_weight_adjustment(summary)
    fdi_adj.to_csv(
        f"/artifacts/retention/fdi_adjustment_{run_date.isoformat()}.csv",
        index=False,
    )
```

6.5 Use Case — FDI Weighting Pipeline

The output of `fdi_weight_adjustment()` feeds back into Part 2's ML scoring retraining:

1. **Monthly retention run** computes 24-month retention per persona.
2. The `fdi_multiplier` column is upserted into `dim_persona_fdi_weights` (effective date = next Monday).
3. Part 2's training pipeline joins `dim_persona_fdi_weights` to compute sample weights during model training — leads from high-retention personas receive higher gradient signal.
4. QIC day 7 review confirms the multiplier deltas before promotion to production scoring.

6.6 Part 6 Compliance Notes

- `persona_id_at_first_funded` is **immutable** — a borrower's persona is locked at the moment of first funding and never re-derived, even if subsequent behavior would re-classify them. This preserves auditability of retention curves over time.
- Right-censored cohorts are flagged (`is_censored = TRUE`) and excluded from `persona_retention_summary()` aggregations to avoid biasing retention rates downward.
- No protected-class attributes are used as cohort dimensions.

Part 7 — Persona FDI Drift Detection (Statistical Process Control)

7.1 Purpose

Once a persona's FDI (Funnel Desirability Index) prediction is in production, the **observed** realized-FDI will drift away from the **predicted** FDI due to:

- Creative fatigue (hooks stop resonating)
- Persona definition drift (audience composition shifts)
- Macro changes (interest rate moves change borrower intent)
- Scoring model staleness (Part 2 retrain overdue)

Statistical Process Control (SPC) gives us a non-parametric, low-latency early-warning system. We use **two Western Electric rules**:

1. **Rule 1 (point outside 3-sigma)**: any single weekly residual (observed – predicted FDI) outside $[LCL, UCL] = \text{mean} \pm 3\sigma$.
2. **Rule 5 (8 consecutive points on one side of center)**: 8 weeks in a row above (or below) the center line — a sustained small shift that Rule 1 will miss.

When either rule fires, the persona is flagged for QIC review and the scoring model is queued for an off-cycle retrain (Part 2).

7.2 Data Contract

- **Granularity**: weekly per persona.
- **Predicted FDI**: the model's expected Tier-A-or-B routing rate for the persona, computed at lead-generation time and averaged across the week.
- **Observed FDI**: the realized Tier-A-or-B routing rate for that persona's leads, computed after the lead-classification SLA (T+7 days).
- **Residual**: `observed_fdi - predicted_fdi`.
- **Warmup**: 12 weeks of residual history required to compute stable control limits.

7.3 Production Code

```
"""
Persona FDI drift detection via statistical process control (SPC).

Uses Western Electric rules:
  Rule 1: any point outside 3-sigma control limits
  Rule 5: 8 consecutive points on the same side of the center line
"""
```

```

from __future__ import annotations

import logging
from dataclasses import dataclass, field
from datetime import date, datetime, timedelta
from typing import Optional

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sqlalchemy import create_engine, text
from sqlalchemy.engine import Engine

logger = logging.getLogger(__name__)

WARMUP_WEEKS: int = 12
SIGMA_MULTIPLIER: float = 3.0
RUN_LENGTH: int = 8 # Rule 5

@dataclass
class ControlLimits:
    center: float
    sigma: float
    ucl: float # upper control limit (center + 3*sigma)
    lcl: float # lower control limit (center - 3*sigma)

    def to_dict(self) -> dict:
        return {
            "center": round(self.center, 6),
            "sigma": round(self.sigma, 6),
            "ucl": round(self.ucl, 6),
            "lcl": round(self.lcl, 6),
        }

@dataclass
class DriftSignal:
    persona_id: str
    week_start: date
    rule_fired: str # "RULE1_OOC" | "RULE5_RUN" | "NONE"
    residual: float
    center: float
    ucl: float
    lcl: float
    run_side: Optional[str] = None # "UPPER" | "LOWER" for Rule 5
    run_length: int = 0

    def to_dict(self) -> dict:
        return {
            "persona_id": self.persona_id,
            "week_start": self.week_start.isoformat(),
            "rule_fired": self.rule_fired,
            "residual": round(self.residual, 6),
            "center": round(self.center, 6),
            "ucl": round(self.ucl, 6),
            "lcl": round(self.lcl, 6),
            "run_side": self.run_side,
            "run_length": self.run_length,
        }

class FDIDriftDetector:
    """SPC drift detector for per-persona FDI residuals."""

```

```

def __init__(self, warehouse_uri: str) -> None:
    self.engine: Engine = create_engine(warehouse_uri)

# ----- #
# Data fetch
# ----- #
def _fetch_residuals(
    self,
    persona_id: str,
    as_of: date,
    lookback_weeks: int = 26,
) -> pd.DataFrame:
    sql = text(
        """
        SELECT
            DATE_TRUNC('week', w.week_start)::date AS week_start,
            w.persona_id,
            w.predicted_fdi,
            w.observed_fdi,
            w.observed_fdi - w.predicted_fdi AS residual,
            w.lead_volume
        FROM fact_persona_fdi_weekly w
        WHERE w.persona_id = :persona_id
            AND w.week_start >= :start
            AND w.week_start <= :as_of
            AND w.lead_volume >= 25 -- ignore low-volume weeks
        ORDER BY w.week_start;
        """
    )
    start = as_of - timedelta(weeks=lookback_weeks)
    with self.engine.connect() as conn:
        df = pd.read_sql(
            sql, conn,
            params={"persona_id": persona_id, "start": start, "as_of": as_of},
        )
    if df.empty:
        raise ValueError(
            f"No FDI residual history for persona={persona_id} "
            f"in last {lookback_weeks} weeks."
        )
    return df

# ----- #
# Control limits: center = mean residual, sigma = sample std
# ----- #
def compute_control_limits(self, residuals: pd.Series) -> ControlLimits:
    if len(residuals) < WARMUP_WEEKS:
        raise ValueError(
            f"Need >= {WARMUP_WEEKS} weeks of history to compute "
            f"control limits (got {len(residuals)})."
        )
    # Use median for robustness against outliers in warmup window
    center = float(residuals.median())
    sigma = float(residuals.std(ddof=1))
    if sigma <= 0 or not np.isfinite(sigma):
        # Degenerate case: all residuals identical. Use a tiny sigma
        # so the limits are non-trivial but Rule 1 won't fire spuriously.
        sigma = 1e-6
    ucl = center + SIGMA_MULTIPLIER * sigma
    lcl = center - SIGMA_MULTIPLIER * sigma
    return ControlLimits(center=center, sigma=sigma, ucl=ucl, lcl=lcl)

# ----- #
# Rule 1: any point outside [LCL, UCL]
# ----- #

```

```

def check_drift(
    self,
    residuals: pd.Series,
    limits: ControlLimits,
) -> list[int]:
    """Return indices of points outside the 3-sigma band."""
    ooc_mask = (residuals > limits.ucl) | (residuals < limits.lcl)
    return list(residuals.index[ooc_mask])

# ----- #
# Rule 5: 8 consecutive points above (or below) center
# ----- #
def check_trend(
    self,
    residuals: pd.Series,
    limits: ControlLimits,
) -> list[tuple[int, str, int]]:
    """
    Return list of (index, side, run_length) for any run >= RUN_LENGTH.
    Side is 'UPPER' or 'LOWER'.
    """
    above = (residuals > limits.center).astype(int)
    # Identify runs
    signals: list[tuple[int, str, int]] = []
    run_val = 0
    run_len = 0
    prev = -1
    for idx, val in above.items():
        if val == prev:
            run_len += 1
        else:
            run_len = 1
            prev = val
        if run_len >= RUN_LENGTH:
            side = "UPPER" if val == 1 else "LOWER"
            signals.append((idx, side, run_len))
    return signals

# ----- #
# End-to-end run
# ----- #
def run(
    self,
    persona_id: str,
    as_of: date,
) -> tuple[pd.DataFrame, ControlLimits, list[DriftSignal]]:
    """Run drift detection for one persona as of a given date."""
    df = self._fetch_residuals(persona_id, as_of)
    df["week_start"] = pd.to_datetime(df["week_start"]).dt.date

    # Compute control limits on the warmup window (all but last 4 weeks)
    warmup = df.iloc[:-4] if len(df) > 16 else df
    limits = self.compute_control_limits(warmup["residual"])

    # Evaluate drift on full history (warmup + recent)
    ooc_indices = self.check_drift(df["residual"], limits)
    trend_signals = self.check_trend(df["residual"], limits)

    signals: list[DriftSignal] = []
    for idx in ooc_indices:
        row = df.loc[idx]
        signals.append(
            DriftSignal(
                persona_id=persona_id,
                week_start=row["week_start"],
            )

```

```

        rule_fired="RULE1_OOC",
        residual=float(row["residual"]),
        center=limits.center,
        ucl=limits.ucl,
        lcl=limits.lcl,
    )
)
for idx, side, run_len in trend_signals:
    row = df.loc[idx]
    signals.append(
        DriftSignal(
            persona_id=persona_id,
            week_start=row["week_start"],
            rule_fired="RULE5_RUN",
            residual=float(row["residual"]),
            center=limits.center,
            ucl=limits.ucl,
            lcl=limits.lcl,
            run_side=side,
            run_length=run_len,
        )
    )

return df, limits, signals

# ----- #
# Visualization
# ----- #
def plot_control_chart(
    self,
    df: pd.DataFrame,
    limits: ControlLimits,
    signals: list[DriftSignal],
    output_path: str,
    persona_id: str,
) -> str:
    """Render an SPC control chart for the persona."""
    fig, ax = plt.subplots(figsize=(12, 5.5), dpi=140)

    weeks = pd.to_datetime(df["week_start"])
    ax.plot(weeks, df["residual"], marker="o", color="#1f77b4",
            linewidth=1.5, label="Residual (obs - pred FDI)")

    ax.axhline(limits.center, color="#2ca02c", linestyle="-",
               linewidth=1.2, label=f"Center = {limits.center:.4f}")
    ax.axhline(limits.ucl, color="#d62728", linestyle="--",
               linewidth=1.0, label=f"UCL = {limits.ucl:.4f} (+3σ)")
    ax.axhline(limits.lcl, color="#d62728", linestyle="--",
               linewidth=1.0, label=f"LCL = {limits.lcl:.4f} (-3σ)")
    # ±1σ and ±2σ shading for visual reference
    for k, alpha in [(1, 0.08), (2, 0.05)]:
        ax.axhspan(
            limits.center - k * limits.sigma,
            limits.center + k * limits.sigma,
            color="#7f7f7f", alpha=alpha,
        )

    # Highlight signals
    for sig in signals:
        wk = pd.to_datetime(sig.week_start)
        if sig.rule_fired == "RULE1_OOC":
            ax.scatter([wk], [sig.residual], s=120, facecolors="none",
                       edgecolors="#d62728", linewidths=2.0, zorder=5,
                       label="Rule 1 (OOC)" if sig is signals[0] else None)
        elif sig.rule_fired == "RULE5_RUN":

```

```

        ax.scatter([wk], [sig.residual], s=120, marker="s",
                   facecolors="#ff7f0e", edgecolors="black",
                   linewidths=0.8, zorder=5,
                   label="Rule 5 (8-in-a-row)" if sig is signals[0] else None)

    ax.set_title(f"FDI Drift SPC Chart - {persona_id}")
    ax.set_xlabel("Week")
    ax.set_ylabel("Residual (observed - predicted FDI)")
    ax.grid(True, alpha=0.3)
    ax.legend(loc="best", fontsize=8, framealpha=0.9)
    fig.autofmt_xdate()
    fig.tight_layout()
    fig.savefig(output_path, format="png")
    plt.close(fig)
    return output_path

# ----- #
# Persist signals
# ----- #
def persist_signals(self, signals: list[DriftSignal]) -> None:
    if not signals:
        return
    sql = text(
        """
        INSERT INTO fact_fdi_drift_signals
        (persona_id, week_start, rule_fired, residual,
         center, ucl, lcl, run_side, run_length, detected_at)
        VALUES
        (:persona_id, :week_start, :rule_fired, :residual,
         :center, :ucl, :lcl, :run_side, :run_length, NOW());
        """
    )
    with self.engine.begin() as conn:
        for s in signals:
            d = s.to_dict()
            conn.execute(sql, params={
                "persona_id": d["persona_id"],
                "week_start": d["week_start"],
                "rule_fired": d["rule_fired"],
                "residual": d["residual"],
                "center": d["center"],
                "ucl": d["ucl"],
                "lcl": d["lcl"],
                "run_side": d["run_side"],
                "run_length": d["run_length"],
            })

```

7.4 Weekly orchestrator

```

"""Weekly persona FDI drift scan - runs every Monday 06:00 UTC."""
from datetime import date, timedelta

from analytics.fdi_drift import FDI Drift Detector

PERSONAS_TO_MONITOR = [
    "NP-001", "NP-002", "NP-003", "NP-004", "NP-005", "NP-006",
    "NP-007", "NP-009", "NP-010", "NP-012",
    # NP-008 (unpermitted-ADU) and NP-011 (zero-reserves) excluded:
    # these are Decline personas with no Tier-A-or-B routing expected.
]

def weekly_drift_scan(run_date: date) -> None:

```

```

detector = FDIDriftDetector(
    warehouse_uri="postgresql://warehouse_user@wh-prod/analytics"
)
all_signals = []
for persona_id in PERSONAS_TO_MONITOR:
    try:
        df, limits, signals = detector.run(persona_id, run_date)
        if signals:
            all_signals.extend(signals)
            detector.plot_control_chart(
                df, limits, signals,
                output_path=f"/artifacts/fdi_drift/{persona_id}_{run_date}.png",
                persona_id=persona_id,
            )
    except ValueError as e:
        # Persona in warmup or insufficient history – skip silently
        continue
    except Exception:
        # Log and continue; one persona failure shouldn't break the scan
        import logging
        logging.getLogger(__name__).exception(
            "FDI drift scan failed for %s", persona_id
        )

if all_signals:
    detector.persist_signals(all_signals)
# Page on-call if any Rule 1 signal
rule1 = [s for s in all_signals if s.rule_fired == "RULE1_OOC"]
if rule1:
    # alerting integration (PagerDuty, Slack, etc.)
    pass

```

7.5 Part 7 Compliance Notes

- Drift detection is **persona-level**, not borrower-level. No protected-class attributes are inputs to the residual computation.
- Rule 5 catches the failure mode where creative fatigue slowly degrades observed FDI by 0.5%/week for 8+ weeks — a shift too small for Rule 1 to catch but cumulatively material.
- When a signal fires, the persona is **not** automatically re-scored. The QIC day 7 review confirms the cause (creative fatigue vs. scoring staleness vs. persona drift) before promoting an off-cycle Part 2 retrain.

Part 8 — LTV/CAC Modeling per Persona

8.1 Purpose

LTV/CAC is the **unit economics** gate. A persona is investable only if $LTV / CAC \geq 3.0$ — meaning the lifetime value of a funded borrower in that persona exceeds the cost to acquire them by at least 3×. Below 3.0, the persona's paid-spend allocation is reduced; below 1.5, the persona is paused entirely (mirrors Part 5's ROAS gate at the persona level).

8.2 Definitions

8.2.1 LTV (Lifetime Value)

For a single funded borrower in persona P:

```
LTV_P = origination_fee_P
        + E_t [interest_revenue_P(t) × survival_P(t) × discount_factor^t]
        + repeat_borrower_LTV_P × retention_24m_P
```

Where:

- **origination_fee_P**: average origination fee at first funding for persona P.
- **interest_revenue_P(t)**: monthly interest revenue, modeled as a step function with early-payoff hazard.
- **survival_P(t)**: probability the loan is still outstanding at month t (Kaplan-Meier by persona).
- **discount_factor^t**: NPV discount (monthly rate $r = 0.0083$, annual $\sim 10\%$).
- **repeat_borrower_LTV_P**: LTV of a second-loan borrower (same formula, second origination).
- **retention_24m_P**: 24-month repeat-borrowing retention rate from Part 6.

8.2.2 CAC (Customer Acquisition Cost)

For persona P:

```
CAC_P = (ad_spend_P + LO_time_cost_P + onboarding_cost_P)
        / funded_loans_P
```

Where:

- **ad_spend_P**: paid-media spend attributed to leads tagged persona P (last-touch within 30 days).
- **LO_time_cost_P**: LO hours × loaded hourly rate, allocated by persona mix.
- **onboarding_cost_P**: fixed cost per funded loan (KYC, underwriting review, doc prep).
- **funded_loans_P**: count of loans funded for persona P in the same period.

8.3 Production Code

```
"""
Per-persona LTV/CAC model.

LTV = origination_fee + NPV(interest_revenue * survival) +
      repeat_borrower_LTV * retention_24m
CAC = (ad_spend + LO_time_cost + onboarding_cost) / funded_loans
"""
from __future__ import annotations
```

```

import logging
from dataclasses import dataclass
from datetime import date
from typing import Optional

import numpy as np
import pandas as pd
from sqlalchemy import create_engine, text
from sqlalchemy.engine import Engine

logger = logging.getLogger(__name__)

# Discount rate: 10% annual → monthly
ANNUAL_DISCOUNT_RATE: float = 0.10
MONTHLY_DISCOUNT_RATE: float = (1 + ANNUAL_DISCOUNT_RATE) ** (1 / 12) - 1
LOAN_HOLD_MONTHS: int = 24 # max horizon for NPV calc

LTV_CAC_HEALTHY: float = 3.0
LTV_CAC_MINIMUM: float = 1.5 # below this → pause persona

@dataclass
class PersonaUnitEconomics:
    persona_id: str
    origination_fee: float
    interest_npv: float
    repeat_borrower_ltv: float
    retention_24m: float
    ltv: float
    ad_spend: float
    lo_time_cost: float
    onboarding_cost: float
    funded_loans: int
    cac: float
    ltv_cac_ratio: float
    health_flag: str # "HEALTHY" | "WATCH" | "PAUSE"

    def to_dict(self) -> dict:
        return {
            "persona_id": self.persona_id,
            "origination_fee": round(self.origination_fee, 2),
            "interest_npv": round(self.interest_npv, 2),
            "repeat_borrower_ltv": round(self.repeat_borrower_ltv, 2),
            "retention_24m": round(self.retention_24m, 4),
            "ltv": round(self.ltv, 2),
            "ad_spend": round(self.ad_spend, 2),
            "lo_time_cost": round(self.lo_time_cost, 2),
            "onboarding_cost": round(self.onboarding_cost, 2),
            "funded_loans": int(self.funded_loans),
            "cac": round(self.cac, 2),
            "ltv_cac_ratio": round(self.ltv_cac_ratio, 4),
            "health_flag": self.health_flag,
        }

class LTVCACModeler:
    """Per-persona LTV/CAC computation."""

    def __init__(self, warehouse_uri: str) -> None:
        self.engine: Engine = create_engine(warehouse_uri)

    # ----- #
    # Survival curve (Kaplan-Meier) per persona — precomputed in warehouse
    # ----- #
    def _fetch_survival_curves(self) -> pd.DataFrame:

```

```

    """Returns DataFrame: persona_id | month | survival_prob."""
    sql = text(
        """
        SELECT persona_id, month_offset AS month, survival_prob
        FROM mart_persona_survival_curves
        WHERE month_offset BETWEEN 0 AND :max_month
        ORDER BY persona_id, month_offset;
        """
    )
    with self.engine.connect() as conn:
        df = pd.read_sql(sql, conn, params={"max_month": LOAN_HOLD_MONTHS})
    return df

# ----- #
# Persona-level inputs
# ----- #
def _fetch_persona_inputs(self, as_of: date, lookback_days: int = 90) -> pd.DataFrame:
    sql = text(
        """
        SELECT
            p.persona_id,
            p.avg_origination_fee,
            p.avg_monthly_interest_revenue,
            p.retention_24m,
            p.repeat_borrower_ltv,
            COALESCE(s.ad_spend, 0) AS ad_spend,
            COALESCE(s.lo_time_cost, 0) AS lo_time_cost,
            COALESCE(s.onboarding_cost, 0) AS onboarding_cost,
            COALESCE(s.funded_loans, 0) AS funded_loans
        FROM dim_personas p
        LEFT JOIN agg_persona_spend_90d s
            ON s.persona_id = p.persona_id
            AND s.as_of_date = :as_of
        WHERE p.is_active = TRUE;
        """
    )
    with self.engine.connect() as conn:
        df = pd.read_sql(sql, conn, params={"as_of": as_of})
    return df

# ----- #
# NPV of interest revenue * survival
# ----- #
@staticmethod
def _compute_interest_npv(
    avg_monthly_interest: float,
    survival: pd.Series,
    months: pd.Series,
) -> float:
    """ $\sum_t [\text{interest}(t) * \text{survival}(t) / (1 + r)^t]$ """
    if survival.empty:
        return 0.0
    discount_factors = 1.0 / (1.0 + MONTHLY_DISCOUNT_RATE) ** months.values
    return float(np.sum(avg_monthly_interest * survival.values * discount_factors))

# ----- #
# Health classification
# ----- #
@staticmethod
def _classify(ltv_cac: float) -> str:
    if ltv_cac >= LTV_CAC_HEALTHY:
        return "HEALTHY"
    if ltv_cac >= LTV_CAC_MINIMUM:
        return "WATCH"
    return "PAUSE"

```

```

# ----- #
# End-to-end run
# ----- #
def compute_all(self, as_of: date) -> list[PersonaUnitEconomics]:
    inputs = self._fetch_persona_inputs(as_of)
    if inputs.empty:
        return []
    survival = self._fetch_survival_curves()

    results: list[PersonaUnitEconomics] = []
    for _, row in inputs.iterrows():
        persona_id = row["persona_id"]
        surv_p = survival[survival["persona_id"] == persona_id]
        if surv_p.empty:
            logger.warning("No survival curve for %s - skipping.", persona_id)
            continue

        interest_npv = self._compute_interest_npv(
            avg_monthly_interest=float(row["avg_monthly_interest_revenue"]),
            survival=surv_p["survival_prob"],
            months=surv_p["month"],
        )

        retention_24m = float(row["retention_24m"])
        repeat_ltv = float(row["repeat_borrower_ltv"])
        origination_fee = float(row["avg_origination_fee"])

        ltv = (
            origination_fee
            + interest_npv
            + repeat_ltv * retention_24m
        )

        funded = int(row["funded_loans"])
        if funded <= 0:
            cac = float("inf")
            ltv_cac = 0.0
            health = "PAUSE"
        else:
            total_cost = (
                float(row["ad_spend"])
                + float(row["lo_time_cost"])
                + float(row["onboarding_cost"])
            )
            cac = total_cost / funded
            ltv_cac = ltv / cac if cac > 0 else float("inf")
            health = self._classify(ltv_cac)

        results.append(
            PersonaUnitEconomics(
                persona_id=persona_id,
                origination_fee=origination_fee,
                interest_npv=interest_npv,
                repeat_borrower_ltv=repeat_ltv,
                retention_24m=retention_24m,
                ltv=ltv,
                ad_spend=float(row["ad_spend"]),
                lo_time_cost=float(row["lo_time_cost"]),
                onboarding_cost=float(row["onboarding_cost"]),
                funded_loans=funded,
                cac=cac,
                ltv_cac_ratio=ltv_cac,
                health_flag=health,
            )
        )

```

```

    return results

# ----- #
# Persistence
# ----- #
def persist(self, results: list[PersonaUnitEconomics], as_of: date) -> None:
    if not results:
        return
    sql = text(
        """
        INSERT INTO fact_persona_ltv_cac
        (persona_id, as_of_date, origination_fee, interest_npv,
         repeat_borrower_ltv, retention_24m, ltv, ad_spend,
         lo_time_cost, onboarding_cost, funded_loans, cac,
         ltv_cac_ratio, health_flag)
        VALUES
        (:persona_id, :as_of, :of, :inpv, :rbl, :r24,
         :ltv, :ads, :lotc, :obc, :fl, :cac, :ratio, :flag);
        """
    )
    with self.engine.begin() as conn:
        for r in results:
            d = r.to_dict()
            conn.execute(sql, params={
                "persona_id": d["persona_id"],
                "as_of": as_of,
                "of": d["origination_fee"],
                "inpv": d["interest_npv"],
                "rbl": d["repeat_borrower_ltv"],
                "r24": d["retention_24m"],
                "ltv": d["ltv"],
                "ads": d["ad_spend"],
                "lotc": d["lo_time_cost"],
                "obc": d["onboarding_cost"],
                "fl": d["funded_loans"],
                "cac": d["cac"],
                "ratio": d["ltv_cac_ratio"],
                "flag": d["health_flag"],
            })

```

8.4 Persona-by-Persona LTV/CAC Table

The table below is regenerated quarterly and lives in `fact_persona_ltv_cac`. Values shown are illustrative targets used in QBR planning (Part 12).

Persona ID	Description	Avg Orig Fee	Interest NPV (24m)	Repeat LTV × Retention	LTV	CAC	LTV/CAC	Health
NP-001	Cash-out refi, prime	\$4,200	\$9,800	\$7,300 × 0.34	\$16,482	\$3,900	4.23	HEALTHY
NP-002	Cash-out refi, near-prime	\$4,000	\$8,400	\$6,500 × 0.28	\$10,220	\$4,100	2.49	WATCH
NP-003	DSCR investor, established	\$5,800	\$14,200	\$11,400 × 0.41	\$26,474	\$5,200	5.09	HEALTHY

Persona ID	Description	Avg Orig Fee	Interest NPV (24m)	Repeat LTV × Retention	LTV	CAC	LTV/CAC	Health
NP-004	DSCR investor, emerging	\$5,200	\$10,900	\$9,200 × 0.22	\$13,124	\$4,800	2.73	WATCH
NP-005	Bridge-to-rent, experienced	\$5,500	\$11,300	\$10,100 × 0.31	\$18,911	\$4,600	4.11	HEALTHY
NP-006	Bridge-to-rent, first-time	\$5,000	\$9,400	\$8,100 × 0.19	\$9,939	\$5,000	1.99	WATCH
NP-007	Fix-and-flip, licensed GC	\$6,200	\$12,800	\$10,900 × 0.36	\$22,924	\$5,500	4.17	HEALTHY
NP-009	Fix-and-flip, first-time	\$5,900	\$9,800	\$9,400 × 0.17	\$10,898	\$5,900	1.85	WATCH
NP-010	ADU-permitted, SFD	\$4,800	\$10,200	\$8,200 × 0.29	\$17,378	\$4,400	3.95	HEALTHY
NP-012	ADU-permitted, multi-fam	\$4,600	\$9,700	\$7,800 × 0.24	\$16,172	\$4,500	3.59	HEALTHY
NP-008	ADU-unpermitted	\$3,800	\$4,100	\$2,900 × 0.08	\$4,132	\$3,200	1.29	PAUSE
NP-011	Zero-reserves	\$3,500	\$2,200	\$1,400 × 0.04	\$1,756	\$3,100	0.57	PAUSE

Summary: 6 personas HEALTHY, 4 WATCH, 2 PAUSE. The PAUSE personas (NP-008, NP-011) align with the TS-10 Part 2A contract — these are Decline-routed personas with no expected Tier-A-or-B outcome. Their paid-spend allocation is zero by policy.

8.5 Part 8 Compliance Notes

- LTV/CAC is computed at the **persona** level, never the individual borrower level — protects against disparate-impact risk in pricing decisions.
- The "Repeat LTV × Retention" component uses the persona-level retention from Part 6, which is itself computed without protected-class features.
- The PAUSE flag for NP-008 and NP-011 is **not** a fair-lensing concern: these personas are paused because they cannot route to Tier A or B (TS-10 Part 2A contract), not because of borrower demographics. The fair-lensing audit (Part 12) confirms this quarterly.

Part 9 — Data Warehouse Schema (dbt Star Schema)

9.1 Design Principles

- **Star schema** with conformed dimensions shared across fact tables.
- **Surrogate keys** (hash-based) on every dimension for join stability.
- **Type-2 SCDs** for `dim_borrowers` (persona re-assignment creates a new row).
- **Materialization policy**: staging models = `view`; dimension models = `table`; fact models = `incremental`.
- **PII isolation**: PII columns live in a separate `dim_borrowers_pii` model, granted only to compliance role.
- **Tests**: every primary key has `unique + not_null`; every foreign key has `relationships`.

9.2 Project Layout

```

dbt/
├─ dbt_project.yml
├─ profiles.yml
├─ models/
│   ├─ staging/
│   │   ├─ _sources.yml
│   │   ├─ stg_leads.sql
│   │   ├─ stg_borrowers.sql
│   │   ├─ stg_loans.sql
│   │   ├─ stg_properties.sql
│   │   ├─ stg_lenders.sql
│   │   └─ stg_ad_spend.sql
│   ├─ intermediate/
│   │   ├─ int_lead_classifications.sql
│   │   └─ int_persona_assignments.sql
│   └─ marts/
│       ├─ core/
│       │   ├─ fct_leads.sql
│       │   ├─ fct_loans.sql
│       │   ├─ fct_ad_spend.sql
│       │   ├─ fct_bandit_snapshots.sql
│       │   ├─ dim_borrowers.sql
│       │   ├─ dim_properties.sql
│       │   ├─ dim_lenders.sql
│       │   ├─ dim_personas.sql
│       │   └─ dim_channels.sql
│       │   └─ _core.yml
│       └─ pii/
│           └─ dim_borrowers_pii.sql
└─ macros/
    └─ hash_surrogate_key.sql

```

9.3 dbt_project.yml

```

name: 'lending_warehouse'
version: '1.4.0'
config-version: 2

```

```

profile: 'lending_warehouse'

model-paths: ["models"]
seed-paths:  ["seeds"]
macro-paths: ["macros"]

target-path: "target"
clean-targets:
  - "target"
  - "dbt_packages"

models:
  lending_warehouse:
    staging:
      +materialized: view
      +schema: staging
      +tags: ["staging"]
    intermediate:
      +materialized: view
      +schema: intermediate
      +tags: ["intermediate"]
    marts:
      core:
        +materialized: table
        +schema: core
        +tags: ["core"]
        dim_*:
          +materialized: table
        fct_*:
          +materialized: incremental
          +incremental_strategy: merge
          +unique_key: surrogate_id
          +tags: ["incremental"]
      pii:
        +materialized: table
        +schema: pii
        +tags: ["pii"]
        +post-hook:
          - "grant select on {{ this }} to role_compliance_only"

```

9.4 Staging Models

stg_leads.sql

```

{{ config(materialized='view') }}

with source as (
  select * from {{ source('raw_crm', 'leads') }}
),

renamed as (
  select
    md5(lead_id::text)           as lead_surrogate_id,
    lead_id                    as natural_lead_id,
    borrower_id                 as natural_borrower_id,
    property_id                 as natural_property_id,
    channel_id,
    creative_id,
    dma_id,
    cast(created_at as timestamp) as created_at,
    cast(form_submitted_at as timestamp) as form_submitted_at,

```

```

        cast(qualified_at as timestamp)          as qualified_at,
        utm_source,
        utm_medium,
        utm_campaign,
        landing_page_url,
        is_test_lead
    from source
)

select * from renamed

```

stg_borrowers.sql

```

{{ config(materialized='view') }}

select
    md5(borrower_id::text)          as borrower_surrogate_id,
    borrower_id                    as natural_borrower_id,
    credit_band,
    state_code,
    dma_id,
    annual_income_band,
    reserves_band,
    property_owner_status,
    created_at
from {{ source('raw_crm', 'borrowers') }}
where is_merged = false

```

stg_loans.sql

```

{{ config(materialized='view') }}

select
    md5(loan_id::text)              as loan_surrogate_id,
    loan_id                        as natural_loan_id,
    borrower_id                    as natural_borrower_id,
    property_id                    as natural_property_id,
    lender_id                      as natural_lender_id,
    loan_product,
    loan_amount,
    interest_rate,
    origination_fee,
    cast(applied_at as timestamp)   as applied_at,
    cast(underwritten_at as timestamp) as underwritten_at,
    cast(funded_at as timestamp)    as funded_at,
    cast(paid_off_at as timestamp)  as paid_off_at,
    loan_status,
    loan_number -- 1 = first, 2 = repeat, ...
from {{ source('raw_los', 'loans') }}

```

stg_ad_spend.sql

```

{{ config(materialized='view') }}

select
    md5(channel_id || '|' || dma_id || '|' || spend_date::text) as ad_spend_surrogate_id,
    channel_id,
    creative_id,
    dma_id,

```

```

    cast(spend_date as date)           as spend_date,
    impressions,
    clicks,
    spend_usd,
    platform_reported_conversions
from {{ source('raw_ad_platforms', 'unified_spend') }}

```

9.5 Dimension Models

dim_borrowers.sql (Type-2 SCD)

```

{{ config(materialized='table') }}

with persona_assignments as (
    select * from {{ ref('int_persona_assignments') }}
),

scd as (
    select
        b.borrower_surrogate_id,
        b.natural_borrower_id,
        b.credit_band,
        b.state_code,
        b.dma_id,
        b.annual_income_band,
        b.reserves_band,
        b.property_owner_status,
        pa.persona_id,
        pa.persona_assigned_at      as effective_from,
        coalesce(
            lead(pa.persona_assigned_at) over (
                partition by b.natural_borrower_id
                order by pa.persona_assigned_at
            ),
            '2999-12-31'::timestamp
        ) as effective_to,
        case
            when lead(pa.persona_assigned_at) over (
                partition by b.natural_borrower_id
                order by pa.persona_assigned_at
            ) is null then true
            else false
        end as is_current
    from {{ ref('stg_borrowers') }} b
    left join persona_assignments pa
        on pa.natural_borrower_id = b.natural_borrower_id
)

select * from scd

```

dim_personas.sql

```

{{ config(materialized='table') }}

select
    persona_id,
    persona_name,
    persona_description,
    tier_route_default,      -- 'A', 'B', or 'DECLINE'

```

```

    is_active,
    created_at,
    retired_at
from {{ source('raw_config', 'persona_library') }}
```

dim_properties.sql

```

{{ config(materialized='table') }}

select
    md5(property_id::text)          as property_surrogate_id,
    property_id                    as natural_property_id,
    property_type,                  -- SFR, CONDO, MULTI_2_4, MULTI_5_PLUS, ADU
    occupancy_status,               -- PRIMARY, SECONDARY, INVESTMENT
    state_code,
    dma_id,
    census_tract,                   -- for fair-lensing audit only
    appraised_value,
    year_built
from {{ source('raw_crm', 'properties') }}
```

dim_lenders.sql

```

{{ config(materialized='table') }}

select
    md5(lender_id::text)           as lender_surrogate_id,
    lender_id                      as natural_lender_id,
    lender_name,
    lender_type,                   -- BANK, CREDIT_UNION, PRIVATE, SPECIALTY
    specialty_persona_ids,         -- array of persona_ids this lender accepts
    states_served,
    is_active
from {{ source('raw_config', 'lender_programs') }}
```

dim_channels.sql

```

{{ config(materialized='table') }}

select
    md5(channel_id::text)          as channel_surrogate_id,
    channel_id                    as natural_channel_id,
    channel_name,                  -- 'meta_paid', 'google_paid', 'linkedin_paid', 'organic'
    channel_category,              -- 'PAID_SOCIAL', 'PAID_SEARCH', 'PAID_B2B', 'ORGANIC'
    is_paid,
    is_active
from {{ source('raw_config', 'channels') }}
```

9.6 Fact Models

fct_leads.sql (incremental)

```

{{ config(
    materialized='incremental',
    unique_key='lead_surrogate_id',
```

```

        incremental_strategy='merge',
        tags=['core', 'incremental']
    ) }}

with src as (
    select * from {{ ref('stg_leads') }}

    {% if is_incremental() %}
    where created_at > (select coalesce(max(created_at), '1970-01-01') from {{ this }})
    {% endif %}
),

classifications as (
    select * from {{ ref('int_lead_classifications') }}
),

fct as (
    select
        s.lead_surrogate_id,
        s.natural_lead_id,
        s.natural_borrower_id,
        s.natural_property_id,
        b.persona_id,
        b.borrower_surrogate_id,
        p.property_surrogate_id,
        s.channel_id,
        c.channel_surrogate_id,
        s.creative_id,
        s.dma_id,
        s.created_at,
        s.form_submitted_at,
        s.qualified_at,
        cl.tier_routed,           -- 'A', 'B', 'DECLINE'
        cl.tier_routed_at,
        cl.fdi_score,
        cl.ml_score,
        s.utm_source,
        s.utm_medium,
        s.utm_campaign,
        s.is_test_lead
    from src s
    left join {{ ref('dim_borrowers') }} b
        on b.natural_borrower_id = s.natural_borrower_id
        and b.is_current = true
    left join {{ ref('dim_properties') }} p
        on p.natural_property_id = s.natural_property_id
    left join {{ ref('dim_channels') }} c
        on c.natural_channel_id = s.channel_id
    left join classifications cl
        on cl.natural_lead_id = s.natural_lead_id
)

select * from fct

```

fct_loans.sql

```

{{ config(
    materialized='incremental',
    unique_key='loan_surrogate_id',
    incremental_strategy='merge'
) }}

select

```

```

    l.loan_surrogate_id,
    l.natural_loan_id,
    l.natural_borrower_id,
    l.natural_property_id,
    l.natural_lender_id,
    b.persona_id,
    b.borrower_surrogate_id,
    p.property_surrogate_id,
    len.lender_surrogate_id,
    l.loan_product,
    l.loan_amount,
    l.interest_rate,
    l.origination_fee,
    l.applied_at,
    l.underwritten_at,
    l.funded_at,
    l.paid_off_at,
    l.loan_status,
    l.loan_number
from {{ ref('stg_loans') }} l
left join {{ ref('dim_borrowers') }} b
    on b.natural_borrower_id = l.natural_borrower_id
    and b.is_current = true
left join {{ ref('dim_properties') }} p
    on p.natural_property_id = l.natural_property_id
left join {{ ref('dim_lenders') }} len
    on len.natural_lender_id = l.natural_lender_id
{% if is_incremental() %}
where l.applied_at > (select coalesce(max(applied_at), '1970-01-01') from {{ this }})
{% endif %}

```

fct_ad_spend.sql

```

{{ config(
    materialized='incremental',
    unique_key='ad_spend_surrogate_id',
    incremental_strategy='merge'
) }}

select
    a.ad_spend_surrogate_id,
    a.channel_id,
    c.channel_surrogate_id,
    a.creative_id,
    a.dma_id,
    a.spend_date,
    a.impressions,
    a.clicks,
    a.spend_usd,
    a.platform_reported_conversions
from {{ ref('stg_ad_spend') }} a
left join {{ ref('dim_channels') }} c
    on c.natural_channel_id = a.channel_id
{% if is_incremental() %}
where a.spend_date > (select coalesce(max(spend_date), '1970-01-01') from {{ this }})
{% endif %}

```

fct_bandit_snapshots.sql

```

{{ config(materialized='incremental', unique_key='snapshot_id') }}

```

```

select
  md5(persona_id || '|' || arm_id || '|' || snapshot_at::text) as snapshot_id,
  persona_id,
  arm_id,
  arm_label,
  snapshot_at,
  alpha,
  beta,
  posterior_mean,
  posterior_ci_lower,
  posterior_ci_upper,
  sample_count,
  reward_count,
  is_retired,
  retirement_reason
from {{ source('raw_ml', 'bandit_snapshots') }}

```

9.7 PII-Isolated Model

dim_borrowers_pii.sql

```

{{ config(
  materialized='table',
  schema='pii',
  post_hook="grant select on {{ this }} to role_compliance_only"
) }}

select
  md5(borrower_id::text)           as borrower_surrogate_id,
  borrower_id                     as natural_borrower_id,
  first_name,
  last_name,
  email_sha256,                   -- pre-hashed at ingest
  phone_sha256,                   -- pre-hashed at ingest
  mailing_address_full,
  date_of_birth,
  ssn_last4_hash
from {{ source('raw_crm', 'borrowers_pii') }}

```

9.8 Tests (_core.yml excerpt)

```

version: 2

models:
  - name: fct_leads
    description: "One row per lead."
    columns:
      - name: lead_surrogate_id
        tests: [unique, not_null]
      - name: borrower_surrogate_id
        tests:
          - relationships:
              to: ref('dim_borrowers')
              field: borrower_surrogate_id
      - name: channel_surrogate_id
        tests:
          - relationships:
              to: ref('dim_channels')

```

```

        field: channel_surrogate_id

- name: dim_borrowers
  description: "Type-2 SCD of borrowers."
  columns:
    - name: borrower_surrogate_id
      tests: [not_null]
    - name: persona_id
      tests: [not_null]

- name: fct_ad_spend
  columns:
    - name: ad_spend_surrogate_id
      tests: [unique, not_null]
    - name: spend_usd
      tests:
        - not_null
        - dbt_utils.expression_is_true:
            expression: ">= 0"

```

9.9 Part 9 Compliance Notes

- The `census_tract` column on `dim_properties` is **not** used for any persona scoring or routing decision. It exists only for the Part 12 fair-lensing audit, which computes approval-rate parity across census tracts as a proxy for protected-class fairness.
- PII columns are isolated in a separate schema with explicit grants. Reverse-ETL pipelines (Part 10) read from hashed columns, never raw PII.
- Type-2 SCDs on `dim_borrowers` preserve the historical persona assignment at the moment of lead creation — critical for backtesting Parts 2, 3, 4 against the persona definitions in effect at that time.

Part 10 — Reverse ETL Pipeline (Census/Hightouch Spec)

10.1 Purpose

Reverse ETL moves data **out of** the warehouse **into** operational systems (ad platforms, CRMs, marketing-automation tools). The pattern is: dbt builds clean models → Census/Hightouch syncs them on a schedule → ad platforms consume.

This runbook specifies four sync pipelines that operationalize the persona library, scoring, and compliance gates from Parts 1–9.

10.2 Sync Pipelines Overview

#	Pipeline	Source model	Destination	Cadence	Purpose
1	Funded-loan borrower audience	<code>mart_reverse_etl.funded_loan_borrowers</code>	Meta Custom Audience	Weekly (Sun 02:00 UTC)	Lookalike source for Part 11
2	Tier-A-or-B lead list	<code>mart_reverse_etl.tier_ab_leads</code>	Google Enhanced Conversions	Daily (01:00 UTC)	Online conversion optimization
3	Declined-but-re-engagement-ready	<code>mart_reverse_etl.suppression_list</code>	Meta + Google + LinkedIn suppression	Daily (01:30 UTC)	Prevent re-targeting un-fundable leads
4	Persona-augmented lead list	<code>mart_reverse_etl.leads_with_persona</code>	HubSpot (CRM)	Hourly	Operational LO routing

10.3 PII Handling — SHA-256 Hashing at the Warehouse Layer

All PII columns synced to ad platforms are **SHA-256 hashed in the warehouse dbt model**, never in the reverse-ETL tool. This guarantees:

1. **Single source of truth** for hashing logic (one dbt macro, audited in git).
2. **No raw PII** ever leaves the warehouse boundary.
3. **Deterministic** across syncs — same email always hashes to the same value, enabling cross-platform identity resolution.

Hashing macro

```
-- macros/sha256_hash.sql
{% macro sha256_hash(column_name) %}
    encode(digest(cast('{{ column_name }} as text), 'sha256'), 'hex')
{% endmacro %}
```

Usage in a dbt model

```
-- models/marts/reverse_etl/funded_loan_borrowers.sql
{{ config(materialized='table', schema='mart_reverse_etl') }}
```

```
select
    {{ sha256_hash('b.email') }}          as email_hash,
    {{ sha256_hash('b.phone') }}          as phone_hash,
    {{ sha256_hash("md5(b.first_name || b.last_name || b.zip)") }}
                                          as name_zip_hash,
    b.borrower_id                         as internal_id,
    b.persona_id,
    max(fl.funded_at)                     as most_recent_funded_at,
    count(distinct fl.loan_id)             as funded_loan_count
from {{ ref('dim_borrowers') }} b
join {{ ref('fct_loans') }} fl
    on fl.borrower_surrogate_id = b.borrower_surrogate_id
where fl.loan_status = 'FUNDED'
    and b.is_current = true
```

```

    and fl.funded_at >= dateadd('month', -24, current_date)
group by 1, 2, 3, 4, 5

```

10.4 Sync Models (dbt)

10.4.1 mart_reverse_etl.funded_loan_borrowers

```

{{ config(materialized='table') }}

select
    {{ sha256_hash('b.email') }}          as email_hash,
    {{ sha256_hash('b.phone') }}          as phone_hash,
    {{ sha256_hash("b.first_name || b.last_name || b.zip") }} as fn_ln_zip_hash,
    b.borrower_id                         as internal_id,
    b.persona_id,
    max(fl.funded_at)                     as most_recent_funded_at,
    count(distinct fl.loan_id)            as funded_loan_count,
    sum(fl.origination_fee)                as lifetime_origination_fees
from {{ ref('dim_borrowers') }} b
join {{ ref('fct_loans') }} fl
    on fl.borrower_surrogate_id = b.borrower_surrogate_id
where fl.loan_status = 'FUNDED'
    and b.is_current = true
    and fl.funded_at >= dateadd('month', -24, current_date)
group by 1, 2, 3, 4, 5

```

10.4.2 mart_reverse_etl.tier_ab_leads

```

{{ config(materialized='table') }}

select
    {{ sha256_hash('b.email') }}          as email_hash,
    {{ sha256_hash('b.phone') }}          as phone_hash,
    l.lead_id                             as internal_id,
    l.persona_id,
    l.tier_routed,
    l.fdi_score,
    l.ml_score,
    l.created_at                           as lead_created_at,
    l.conversion_value_usd                 -- derived from persona LTV (Part 8)
from {{ ref('fct_leads') }} l
left join {{ ref('dim_borrowers') }} b
    on b.borrower_surrogate_id = l.borrower_surrogate_id
where l.tier_routed in ('A', 'B')
    and l.is_test_lead = false
    and l.created_at >= dateadd('day', -30, current_date)

```

10.4.3 mart_reverse_etl.suppression_list

```

{{ config(materialized='table') }}

-- Declined leads that are NOT re-engagement-ready.
-- Re-engagement-ready is defined as: declined >= 90 days ago AND
-- borrower has improved credit_band OR added reserves since decline.
select
    {{ sha256_hash('b.email') }}          as email_hash,
    {{ sha256_hash('b.phone') }}          as phone_hash,

```

```

    l.lead_id                               as internal_id,
    l.persona_id,                           -- 'DECLINE'
    l.tier_routed,                           as lead_created_at,
    l.created_at                             as suppression_type
    'PERMANENT_SUPPRESSION'
from {{ ref('fct_leads') }} l
left join {{ ref('dim_borrowers') }} b
    on b.borrower_surrogate_id = l.borrower_surrogate_id
where l.tier_routed = 'DECLINE'
    and l.is_test_lead = false
    and l.persona_id in ('NP-008', 'NP-011') -- Decline personas per Part 2A
    and not exists (
        select 1
        from {{ ref('fct_leads') }} l2
        where l2.borrower_surrogate_id = l.borrower_surrogate_id
            and l2.tier_routed in ('A', 'B')
            and l2.created_at > l.created_at
    )
)

```

10.5 Census Configuration Spec

Census syncs are defined as code in `census/syncs.yml`. Below is the full spec.

10.5.1 Funded-loan borrower audience → Meta Custom Audience

```

# census/syncs.yml
syncs:
- name: funded_loan_borrowers_to_meta_lookalike_source
  description: >
    Syncs SHA-256 hashed emails/phones of funded-loan borrowers
    to Meta as a custom audience; this audience is then used as
    the source for the Part 11 lookalike audience.
  schedule:
    type: cron
    expression: "0 2 * * 0" # Sun 02:00 UTC weekly
  source:
    connection: warehouse_prod
    model: mart_reverse_etl.funded_loan_borrowers
    keys:
      - email_hash
      - phone_hash
  destination:
    connection: meta_ads_prod
    object: custom_audience
    audience_id: "${META_FUNDED_LOOKALIKE_SOURCE_AUDIENCE_ID}"
    identifier_mapping:
      email_hash: EMAIL_SHA256
      phone_hash: PHONE_SHA256
    operation: update_users # replace full audience
  field_mapping:
    - source: email_hash
      dest: EMAIL_SHA256
    - source: phone_hash
      dest: PHONE_SHA256
  behavior:
    on_record_change: update
    on_record_delete: remove
  notifications:
    on_success: slack://#data-alerts
    on_failure: pagerduty://PDX-REVERSE-ETL
  sla:

```

```
max_latency_minutes: 60
max_records_missing_percent: 0.1
```

10.5.2 Tier-A-or-B leads → Google Enhanced Conversions

```
- name: tier_ab_leads_to_google_enhanced_conversions
  description: &gt;
    Daily sync of Tier-A-or-B qualified leads to Google Ads Enhanced
    Conversions for online conversion optimization.
  schedule:
    type: cron
    expression: "0 1 * * *" # Daily 01:00 UTC
  source:
    connection: warehouse_prod
    model: mart_reverse_etl.tier_ab_leads
    keys:
      - email_hash
      - phone_hash
  destination:
    connection: google_ads_prod
    object: enhanced_conversions
    conversion_action: "${GOOGLE_CONV_ACTION_LEAD_QUALIFIED}"
    identifier_mapping:
      email_hash: hashed_email
      phone_hash: hashed_phone_number
    operation: upsert
  field_mapping:
    - source: email_hash
      dest: hashed_email
    - source: phone_hash
      dest: hashed_phone_number
    - source: conversion_value_usd
      dest: conversion_value
    - source: lead_created_at
      dest: conversion_time
    - source: internal_id
      dest: order_id
  behavior:
    on_record_change: update
  notifications:
    on_success: slack://#data-alerts
    on_failure: pagerduty://PDX-REVERSE-ETL
  sla:
    max_latency_minutes: 30
```

10.5.3 Suppression list → Meta + Google + LinkedIn

```
- name: suppression_list_to_meta
  description: &gt;
    Daily sync of permanent-suppression list (NP-008, NP-011 declined
    leads) to Meta for ad-platform-level exclusion.
  schedule:
    type: cron
    expression: "30 1 * * *" # Daily 01:30 UTC
  source:
    connection: warehouse_prod
    model: mart_reverse_etl.suppression_list
    keys: [email_hash, phone_hash]
  destination:
    connection: meta_ads_prod
    object: custom_audience
```

```

      audience_id: "${META_SUPPRESSION_AUDIENCE_ID}"
      operation: update_users
    field_mapping:
      - source: email_hash
        dest: EMAIL_SHA256
      - source: phone_hash
        dest: PHONE_SHA256
    notifications:
      on_failure: pagerduty://PDX-REVERSE-ETL

- name: suppression_list_to_google
  schedule:
    type: cron
    expression: "30 1 * * *"
  source:
    connection: warehouse_prod
    model: mart_reverse_etl.suppression_list
    keys: [email_hash]
  destination:
    connection: google_ads_prod
    object: customer_match_audience
    audience_id: "${GOOGLE_SUPPRESSION_AUDIENCE_ID}"
    operation: update_users
  field_mapping:
    - source: email_hash
      dest: hashed_email

- name: suppression_list_to_linkedin
  schedule:
    type: cron
    expression: "30 1 * * *"
  source:
    connection: warehouse_prod
    model: mart_reverse_etl.suppression_list
    keys: [email_hash]
  destination:
    connection: linkedin_ads_prod
    object: matched_audience
    audience_id: "${LINKEDIN_SUPPRESSION_AUDIENCE_ID}"
    operation: update_users
  field_mapping:
    - source: email_hash
      dest: hashedEmail

```

10.5.4 Persona-augmented leads → HubSpot

```

- name: leads_with_persona_to_hubspot
  description: >
    Hourly sync of new leads + their persona assignment to HubSpot for
    LO operational routing.
  schedule:
    type: cron
    expression: "0 * * * *" # Hourly
  source:
    connection: warehouse_prod
    model: mart_reverse_etl.leads_with_persona
    keys: [internal_id]
    incremental_field: lead_created_at
  destination:
    connection: hubspot_prod
    object: contact
    identifier_mapping:
      internal_id: properties.lifetime_lead_id

```

```

    operation: upsert
  field_mapping:
    - source: internal_id
      dest: properties.lifetime_lead_id
    - source: persona_id
      dest: properties.persona_id
    - source: tier_routed
      dest: properties.tier_routed
    - source: fdi_score
      dest: properties.fdi_score
    - source: ml_score
      dest: properties.ml_score
    - source: lead_created_at
      dest: properties.lead_created_at
  behavior:
    on_record_change: update
  notifications:
    on_failure: slack://#data-alerts

```

10.6 Census Connection Setup

```

# census/connections.yml
connections:
  - name: warehouse_prod
    type: snowflake
    config:
      account: "${SNOWFLAKE_ACCOUNT}"
      warehouse: COMPUTE_WH
      database: ANALYTICS
      role: CENSUS_SYNC_ROLE
      auth_type: keypair
      private_key: "${SNOWFLAKE_PRIVATE_KEY}"

  - name: meta_ads_prod
    type: facebook_pages
    config:
      access_token: "${META_SYSTEM_USER_TOKEN}"
      ad_account_id: "${META_AD_ACCOUNT_ID}"

  - name: google_ads_prod
    type: google_ads
    config:
      developer_token: "${GOOGLE_ADS_DEVELOPER_TOKEN}"
      refresh_token: "${GOOGLE_ADS_REFRESH_TOKEN}"
      client_customer_id: "${GOOGLE_ADS_CUSTOMER_ID}"

  - name: linkedin_ads_prod
    type: linkedin_ads
    config:
      access_token: "${LINKEDIN_ACCESS_TOKEN}"
      ad_account_id: "${LINKEDIN_AD_ACCOUNT_ID}"

  - name: hubspot_prod
    type: hubspot
    config:
      api_key: "${HUBSPOT_PRIVATE_APP_TOKEN}"

```

10.7 Validation & Monitoring

Check	Method	Threshold	Action on failure
Sync row count vs source row count	Census built-in	diff < 0.1%	Page on-call; block audience activation
Hashed-column null rate	dbt test on <code>mart_reverse_etl.*</code>	0 nulls	Block sync; alert Data/ML
Audience size delta vs prior sync	Census audit log	< $\pm 20\%$ week-over-week	Alert marketing ops; investigate
PII leakage check (raw email in dest?)	Periodic sweep of destination API	0 raw PII fields	Page Compliance; halt all syncs

10.8 Part 10 Compliance Notes

- The suppression list (Pipeline 3) is **not** a fair-lensing concern: it suppresses leads routed to Decline tiers per the TS-10 Part 2A contract (NP-008, NP-011). The decision to decline is based on objective, non-protected-class criteria (permit status, reserves).
- All PII is hashed at the warehouse layer; ad platforms receive only SHA-256 digests. The raw-to-hash mapping exists in `dim_borrowers_pii` (Part 9) under compliance-only grants.
- The `conversion_value_usd` sent to Google Enhanced Conversions is the persona-level LTV (Part 8), not borrower-level pricing — preventing any individual-pricing signal from flowing to ad platforms.

Part 11 — Lookalike Audience Refresh Cadence

11.1 Purpose

Lookalike audiences are derived from a **source audience** (e.g., funded-loan borrowers) by the ad platform's ML model, which finds users with similar profile signals. Lookalikes decay over time because:

1. The source audience grows and shifts in composition.
2. The platform's user base and feature space evolve.
3. Persona definitions drift (Part 7).

A disciplined refresh and retirement cadence prevents lookalike-quality decay from silently eroding paid-channel performance.

11.2 Refresh Schedule

Lookalike tier	Source audience	Refresh cadence	Max age before mandatory refresh	Owner
1% lookalike	Funded-loan borrowers (Pipeline 1)	Every 30 days	45 days	Marketing Ops
3% lookalike	Funded-loan borrowers	Every 30 days	45 days	Marketing Ops
5% lookalike	Funded-loan borrowers	Every 60 days	75 days	Marketing Ops
Tier-A-or-B lookalike	Tier-A-or-B qualified leads	Every 30 days	45 days	Marketing Ops
Persona-specific lookalikes (per persona, 1%)	Tier-A-or-B leads within persona	Every 60 days	75 days	Marketing Ops + Data/ML

Retire schedule: any lookalike audience with **90 days of zero active spend** OR **90 days of Tier-A-or-B rate < 50% of baseline** is retired.

11.3 Audience Size Targets

Audience	Target size	Floor	Ceiling
1% lookalike (funded source)	2.1M – 3.4M (US, 1% of MAU)	1.5M	5M
3% lookalike	6.3M – 10.2M	5M	15M
5% lookalike	10.5M – 17M	8M	25M
Tier-A-or-B 1% lookalike	1.8M – 3M	1M	5M

Floor/ceiling breaches trigger a QIC-flagged review of source-audience health.

11.4 Quality Monitoring

For each active lookalike audience, track monthly:

- **Reach** (impressions / unique users)
- **Cost per qualified lead** (Tier-A-or-B)
- **Tier-A-or-B rate** = (Tier-A-or-B leads from this audience) / (total leads from this audience)
- **FDI mean** (Part 2 ML score distribution)
- **LTV/CAC** of funded borrowers acquired through this audience (Part 8)

Baseline Tier-A-or-B rate is the rate from organic + branded-search leads. A lookalike is **healthy** if its Tier-A-or-B rate $\geq$ 50% of baseline. Below 50% for two consecutive months → retire.

11.4.1 Monitoring query (dbt model)

```
-- models/marts/lookalike_audience_health.sql
{{ config(materialized='table') }}
```

```
with audience_leads as (
    select
        l.lead_id,
        l.created_at,
        l.tier_routed,
        l.audience_id,          -- tagged at ad-set level, flows into utm_campaign
        l.persona_id,
        l.fdi_score,
        date_trunc('month', l.created_at) as month
    from {{ ref('fct_leads') }} l
    where l.audience_id is not null
        and l.is_test_lead = false
),

audience_health as (
    select
        audience_id,
        month,
        count(*)                                as total_leads,
        count(*) filter (where tier_routed in ('A','B')) as tier_ab_leads,
        count(*) filter (where tier_routed in ('A','B')) * 1.0
            / nullif(count(*), 0)                as tier_ab_rate,
        avg(fdi_score)                          as avg_fdi
    from audience_leads
    group by 1, 2
)

select * from audience_health
```

11.5 Audience Lifecycle Documentation Template

Every audience creation, refresh, and retirement is logged in `fact_audience_lifecycle`. Use this template:

```
## Audience Lifecycle Record

- **Audience ID:** [platform audience ID]
- **Audience name:** [human-readable name, e.g. "Funded-Borrower 1% Lookalike v7"]
- **Audience type:** [1% / 3% / 5% lookalike / Tier-AB / persona-specific]
- **Source audience:** [Source audience ID + name]
- **Source audience size at creation:** [N users]
- **Created at:** [timestamp UTC]
- **Created by:** [Marketing Ops engineer]
- **Refresh cycle:** [30d / 60d]
- **Next scheduled refresh:** [date]
- **Persona filter applied:** [persona_id or "ALL"]
- **Estimated size at creation:** [N users]
- **Platform:** [Meta / Google / LinkedIn]

### Refresh log
| Refresh # | Refreshed at | Refreshed by | Source audience size | Lookalike size | Notes |
|---|---|---|---|---|---|

### Retirement log (if retired)
- **Retired at:** [timestamp UTC]
- **Retired by:** [engineer]
```

```
- **Retirement reason:** [zero-spend-90d | quality-decay | persona-retired | source-audience-deprecated]
- **Final Tier-A-or-B rate:** [rate]
- **Final month active:** [YYYY-MM]
```

11.6 Refresh Orchestrator (Python)

```
"""
Lookalike audience refresh orchestrator.
Runs daily at 03:00 UTC; performs refresh for any audience whose
last_refresh_at > refresh_interval_days.
"""

from __future__ import annotations

import logging
from dataclasses import dataclass
from datetime import date, datetime, timedelta
from typing import Optional

from sqlalchemy import create_engine, text
from sqlalchemy.engine import Engine

logger = logging.getLogger(__name__)

# Audience registry: (audience_name, platform, lookalike_pct, refresh_days)
AUDIENCE_REGISTRY = [
    ("funded_borrower_1pct", "meta", 1, 30),
    ("funded_borrower_3pct", "meta", 3, 30),
    ("funded_borrower_5pct", "meta", 5, 60),
    ("tier_ab_1pct", "meta", 1, 30),
    ("persona_np001_1pct", "meta", 1, 60),
    ("persona_np003_1pct", "meta", 1, 60),
    ("persona_np005_1pct", "meta", 1, 60),
    ("persona_np007_1pct", "meta", 1, 60),
    ("persona_np010_1pct", "meta", 1, 60),
    ("persona_np012_1pct", "meta", 1, 60),
    # Tier-A-or-B lookalikes also on Google
    ("tier_ab_1pct_google", "google", 1, 30),
    ("funded_borrower_1pct_google", "google", 1, 30),
]

RETIREMENT_INACTIVITY_DAYS = 90
RETIREMENT_QUALITY_FLOOR_RATIO = 0.50 # 50% of baseline Tier-AB rate

@dataclass
class AudienceStatus:
    audience_name: str
    platform: str
    last_refresh_at: Optional[datetime]
    last_active_spend_at: Optional[datetime]
    last_month_tier_ab_rate: Optional[float]
    baseline_tier_ab_rate: float
    size_estimate: Optional[int]

class LookalikeAudienceManager:
    def __init__(self, warehouse_uri: str, platform_api_keys: dict[str, str]) -> None:
        self.engine = create_engine(warehouse_uri)
        self.platform_api_keys = platform_api_keys

    def _load_status(self) -> list[AudienceStatus]:
        sql = text(
```

```

"""
with latest as (
    select
        audience_name,
        platform,
        max(refreshed_at)           as last_refresh_at,
        max(last_active_spend_at) as last_active_spend_at,
        last_value(tier_ab_rate order by month desc)
            as last_month_tier_ab_rate
    from mart_lookalike_audience_health
    group by 1, 2
),
baseline as (
    select avg(tier_ab_rate) as baseline_rate
    from mart_organic_lead_quality
)
select
    l.audience_name,
    l.platform,
    l.last_refresh_at,
    l.last_active_spend_at,
    l.last_month_tier_ab_rate,
    b.baseline_rate
from latest l
cross join baseline b;
"""

)
with self.engine.connect() as conn:
    rows = conn.execute(sql).fetchall()
return [
    AudienceStatus(
        audience_name=r[0],
        platform=r[1],
        last_refresh_at=r[2],
        last_active_spend_at=r[3],
        last_month_tier_ab_rate=r[4],
        baseline_tier_ab_rate=float(r[5] or 0.0),
        size_estimate=None,
    )
    for r in rows
]

def _due_for_refresh(self, status: AudienceStatus, refresh_days: int) -> bool:
    if status.last_refresh_at is None:
        return True
    return datetime.utcnow() - status.last_refresh_at >= timedelta(days=refresh_days)

def _due_for_retirement(self, status: AudienceStatus) -> tuple[bool, str]:
    if status.last_active_spend_at is not None:
        inactive_days = (datetime.utcnow() - status.last_active_spend_at).days
        if inactive_days >= RETIREMENT_INACTIVITY_DAYS:
            return True, "zero-spend-90d"
    if (
        status.last_month_tier_ab_rate is not None
        and status.baseline_tier_ab_rate > 0
        and status.last_month_tier_ab_rate
            < RETIREMENT_QUALITY_FLOOR_RATIO * status.baseline_tier_ab_rate
    ):
        return True, "quality-decay"
    return False, ""

def _call_platform_refresh(self, status: AudienceStatus) -> int:
    """Stub for platform API call. Returns new size estimate."""
    # Meta: POST /v19.0/{source_audience_id}/lookalikes
    # Google: customer match audience refresh

```

```

    # In production this dispatches to platform-specific SDK calls.
    logger.info("Refreshing audience %s on %s",
               status.audience_name, status.platform)
    return 0 # placeholder

def _call_platform_retire(self, status: AudienceStatus, reason: str) -> None:
    logger.warning("Retiring audience %s on %s (reason=%s)",
                  status.audience_name, status.platform, reason)

def run(self) -> None:
    statuses = self._load_status()
    registry_map = {(name, platform): days
                    for name, platform, _, days in AUDIENCE_REGISTRY}
    for s in statuses:
        key = (s.audience_name, s.platform)
        refresh_days = registry_map.get(key)
        if refresh_days is None:
            continue
        retire, reason = self._due_for_retirement(s)
        if retire:
            self._call_platform_retire(s, reason)
            self._log_retirement(s, reason)
            continue
        if self._due_for_refresh(s, refresh_days):
            new_size = self._call_platform_refresh(s)
            self._log_refresh(s, new_size)

def _log_refresh(self, status: AudienceStatus, new_size: int) -> None:
    sql = text(
        """
        insert into fact_audience_lifecycle
        (audience_name, platform, event_type, event_at, size_estimate)
        values
        (:name, :platform, 'REFRESH', NOW(), :size);
        """
    )
    with self.engine.begin() as conn:
        conn.execute(sql, params={
            "name": status.audience_name,
            "platform": status.platform,
            "size": new_size,
        })

def _log_retirement(self, status: AudienceStatus, reason: str) -> None:
    sql = text(
        """
        insert into fact_audience_lifecycle
        (audience_name, platform, event_type, event_at, reason)
        values
        (:name, :platform, 'RETIRE', NOW(), :reason);
        """
    )
    with self.engine.begin() as conn:
        conn.execute(sql, params={
            "name": status.audience_name,
            "platform": status.platform,
            "reason": reason,
        })

```

11.7 Part 11 Compliance Notes

- Lookalike source audiences are funded-loan borrowers or Tier-A-or-B leads — never based on protected-class attributes.
- Persona-specific lookalikes (e.g., NP-001 lookalike) source from Tier-A-or-B leads **within that persona** — the persona definition itself has been fair-lensed (TS-10 Part 2A).
- Retired audiences are archived, not deleted, to preserve the audit trail. The `fact_audience_lifecycle` table is queryable by Compliance during the Part 12 fair-lensing audit.

Part 12 — QBR Deck Template + Swarm Governance

12.1 Purpose

The Quarterly Business Review (QBR) is the **decision-forcing function** for the iteration cycle. It is held on the last business day of the quarter, attended by the Swarm (Marketing Ops, LO Ops, Compliance, Data/ML, Leadership), and produces a written record of:

1. What was learned this quarter (Parts 2–11 outputs).
2. What will change next quarter (Q+1 plan).
3. What risks are accepted, mitigated, or escalated.

This section defines the deck structure (15 slides) and the Swarm governance RACI for all in-flight processes.

12.2 QBR Deck Structure (15 Slides)

#	Slide	Owner	Source artifacts	Decision type
1	Executive summary	Leadership	All	Inform
2	KPI dashboard	Data/ML	<code>mart_kpi_dashboa</code> <code>rd</code>	Inform
3	Funded-loan cohort analysis	Data/ML	Part 6 cohort retention	Inform + act
4	Cost per funded loan trend	Marketing Ops	<code>fct_ad_spend</code> × <code>fct_loans</code>	Inform + act
5	Persona FDI drift	Data/ML	Part 7 SPC charts	Act
6	Channel mix + ROAS	Marketing Ops	Part 5 CausalImpact decisions	Act
7	Specialty-lender referral performance	LO Ops	<code>fct_loans</code> × <code>dim_lenders</code>	Inform + act

#	Slide	Owner	Source artifacts	Decision type
8	Compliance + fair-lensing update	Compliance	Fair-lensing audit runbook	Act (gating)
9	A/B test learnings	Data/ML	Part 3 Bayesian A/B results	Inform + act
10	Iteration cycle completed	Data/ML	Part 4 bandit + Parts 5–11	Inform
11	Q+1 plan	Leadership	QIC day 30 plan	Act (gating)
12	Risks + mitigations	All	Risk register	Act
13	Decision asks	Leadership	—	Act (gating)
14	Appendix: data warehouse + ML model metrics	Data/ML	dbt docs, ML registry	Inform
15	Appendix: compliance artifacts	Compliance	Fair-lensing reports, audit logs	Inform (audit)

12.2.1 Slide-by-slide content spec

Slide 1 — Executive summary (Leadership)

- 3 KPI headlines: funded-loan count QoQ, cost per funded loan QoQ, LTV/CAC portfolio-weighted QoQ.
- 1 paragraph narrative: top win, top miss, top risk.
- 1 decision ask (cross-references Slide 13).

Slide 2 — KPI dashboard (Data/ML)

- 9-tile grid: leads, Tier-A leads, Tier-B leads, declined, funded loans, funded-loan \$, cost per funded loan, LTV/CAC, ML model AUC.
- Each tile shows QoQ % change and a sparkline of last 4 quarters.
- Color code: green = on target, yellow = within 10% of target, red = >10% off target.

Slide 3 — Funded-loan cohort analysis (Data/ML)

- Part 6 retention curves, one chart per cohort quarter, 4 quarters stacked.
- Table: retention rate at 6, 12, 18, 24 months per persona.
- Callout: personas with >5pp QoQ retention decline.

Slide 4 — Cost per funded loan trend (Marketing Ops)

- Line chart: weekly cost per funded loan, last 13 weeks, by channel.
- Benchmark line: rolling 4-quarter median.
- Callout: channels >15% above benchmark (action: Part 5 holdout review).

Slide 5 — Persona FDI drift (Data/ML)

- SPC charts (Part 7) for any persona with active signal in last 8 weeks.

- One row per persona: chart thumbnail, rule fired, recommended action.
- Decision: which personas get off-cycle Part 2 retrain?

Slide 6 — Channel mix + ROAS (Marketing Ops)

- Part 5 CausalImpact results table: channel, incremental ROAS, CI bounds, decision.
- Sankey: budget allocation last quarter vs. proposed next quarter.

Slide 7 — Specialty-lender referral performance (LO Ops)

- Per-lender: referral volume, conversion-to-funding rate, time-to-funding, repeat-borrow rate.
- Highlight lenders with conversion > benchmark (scale) and < benchmark (outreach).
- Note: specialty lenders serve NP-003, NP-005, NP-007 personas (Part 2A).

Slide 8 — Compliance + fair-lensing update (Compliance)

- Fair-lensing audit results: approval-rate parity by census tract (proxy), by state, by persona.
- Any adverse-impact findings + mitigation plan.
- Compliance attestations signed this quarter.

Slide 9 — A/B test learnings (Data/ML)

- Table of completed Part 3 Bayesian A/B tests: variant, effect size, HDI, decision.
- Tests still in field with expected readout date.

Slide 10 — Iteration cycle completed (Data/ML)

- Timeline: QIC days 1–30 highlights.
- Bandit auto-retirements (Part 4) this quarter.
- Causal-impact decisions (Part 5) this quarter.
- FDI drift signals (Part 7) detected + resolved.
- Reverse-ETL sync uptime (Part 10).

Slide 11 — Q+1 plan (Leadership)

- 3–5 OKRs for next quarter.
- Persona library changes queued (Part 2A).
- Budget reallocation by channel (Part 5 + Part 8).
- New A/B tests queued (Part 3).

Slide 12 — Risks + mitigations (All)

- Risk register top 10, with likelihood × impact matrix.
- Owner + mitigation per risk.

Slide 13 — Decision asks (Leadership)

- 3–5 explicit decisions requiring sign-off in the QBR meeting.
- Each ask: owner, recommendation, alternatives, decision deadline.

Slide 14 — Appendix: data warehouse + ML model metrics (Data/ML)

- dbt test pass rate.
- ML model AUC, calibration, drift (Part 7) per model in production.
- Reverse-ETL sync SLA attainment.

Slide 15 — Appendix: compliance artifacts (Compliance)

- Fair-lensing audit reports (links).
- Compliance attestation log.
- PII access audit (who accessed `dim_borrowers_pii` and why).

12.3 Swarm Governance RACI

RACI legend:

- **R** = Responsible (does the work)
- **A** = Accountable (final approver, one per row)
- **C** = Consulted (input required before decision)
- **I** = Informed (notified after decision)

#	Activity	Marketing Ops	LO Ops	Compliance	Data/ML	Leadership
1	Persona library updates (new persona, retirement, re classification)	C	C	C	R	A
2	Scoring weight changes (Part 2 model retrain promotion)	I	C	C	R	A
3	Creative library updates (new hooks, retired hooks)	R	C	C	C	A
4	Lender program updates (add/remove specialty lenders)	I	R	C	C	A

#	Activity	Marketing Ops	LO Ops	Compliance	Data/ML	Leadership
5	Geo-targeting updates (DMA add/remove, holdout reassignment)	R	C	C	C	A
6	Quarterly iteration cycle (QIC days 1–30 execution)	C	C	C	R	A
7	Annual swarm refresh (full persona + lender library rewrite)	C	C	C	R	A
8	Compliance review (fair-lensing, PII access, attestation)	I	I	R	C	A
9	Fair-lensing audit (census-tract parity, persona parity)	I	I	R	C	A
10	Budget scaling decisions (channel + persona budget deltas > 15%)	R	C	C	C	A
11	A/B test launch approval (Part 3 Bayesian tests)	C	I	C	R	A
12	Bandit arm retirement (Part 4 manual overrides)	C	I	I	R	A
13	Causal-impact holdout design (Part 5 DMA selection)	R	I	C	C	A

#	Activity	Marketing Ops	LO Ops	Compliance	Data/ML	Leadership
14	Reverse-ETL sync additions/removals (Part 10)	R	C	C	C	A
15	Lookalike audience refresh/retire (Part 11)	R	I	C	C	A
16	QBR deck assembly + delivery	C	C	C	R	A
17	Q+1 plan sign-off	C	C	C	C	A
18	Off-cycle model retrain (triggered by Part 7 drift signal)	I	I	C	R	A
19	LTV/CAC threshold changes (Part 8 healthy/minimum targets)	C	C	C	R	A
20	Suppression-list policy changes (Part 10 pipeline 3)	C	I	R	C	A

12.3.1 RACI governance rules

1. **One Accountable per row.** The "A" role is the single approver of record. Decisions are logged with the Accountable's name and date in `fact_governance_decisions`.
2. **Data/ML is R for most technical activities** (rows 2, 6, 7, 11, 12, 16, 18, 19) because the swarm's iteration rhythm is data-led.
3. **Compliance is R for compliance-only activities** (rows 8, 9, 20) — these are non-delegable.
4. **Leadership is A for all activities** — final accountability sits with Leadership, but Leadership does not execute (never R).
5. **Marketing Ops is R for ad-platform operations** (rows 3, 5, 10, 13, 14, 15) — they own the platforms.
6. **LO Ops is R only for lender programs** (row 4) — they own lender relationships.
7. **Consulted before, Informed after.** C roles must be consulted before the decision; I roles are notified after. Violations are logged as governance exceptions.

12.4 Decision Log Schema

Every Swarm governance decision is persisted to `fact_governance_decisions`:

```
create table fact_governance_decisions (
  decision_id          text primary key,
  decision_date        date not null,
  activity_id          int not null,          -- references RACI row #
  decision_summary     text not null,
  decision_detail      text,
  accountable_role     text not null,        -- 'LEADERSHIP'
  accountable_person   text not null,
  consulted_roles      text[],              -- array of roles consulted
  informed_roles       text[],
  effective_date       date not null,
  supersedes_decision_id text,              -- if replacing prior decision
  related_artifacts    jsonb,              -- links to QBR slide, dbt PR, etc.
  created_at          timestamptz default now()
);

create index idx_gov_decisions_activity
  on fact_governance_decisions(activity_id, decision_date desc);
create index idx_gov_decisions_effective
  on fact_governance_decisions(effective_date);
```

12.5 QBR Run-of-Show

Time (mins)	Activity	Owner
0–10	Slide 1 — Executive summary + framing	Leadership
10–25	Slide 2 — KPI dashboard walk-through	Data/ML
25–45	Slides 3–7 — Cohort, CAC, drift, ROAS, lenders	Mixed
45–60	Slide 8 — Compliance + fair-lensing (gating)	Compliance
60–75	Slides 9–10 — A/B + iteration cycle recap	Data/ML
75–105	Slides 11–13 — Q+1 plan, risks, decision asks (gating)	Leadership
105–120	Open discussion + decision sign-offs	All
120+	Appendix walk-through (optional, on-demand)	Data/ML + Compliance

Gating rules:

- Slide 8 must be **passed by Compliance** before Slides 11–13 can be voted on. If Compliance raises an unresolved fair-lensing finding, the Q+1 plan cannot be signed.

- Slide 13 decision asks require unanimous Leadership + Compliance sign-off. Ties go to Compliance (precautionary principle).

12.6 Part 12 Compliance Notes

- The QBR deck is **retained for 7 years** as a compliance artifact (Slide 15 archive). Slides 8 and 13 are signed PDFs with named attestation.
- The RACI is reviewed annually at the Annual Swarm Refresh (row 7). Changes to the RACI itself require Leadership + Compliance joint sign-off.
- The fair-lensing audit (row 9) is non-negotiable: if it is incomplete at QBR time, the QBR is rescheduled, not held without it. This is the single hard gate on the quarterly cycle.

End of Runbook

Parts 1–12 complete.

This runbook is the operating manual for the Quarterly Iteration Cycle. It is versioned in git; every change requires a Swarm RACI-approved PR (rows 6, 7, 16 of Part 12.3). The next quarterly review should re-validate:

- Part 2 ML model AUC and calibration.
- Part 5 CausalImpact holdout design (DMAs may need re-matching as populations shift).
- Part 7 SPC control limits (recomputed every 26 weeks).
- Part 8 LTV/CAC baselines and the $3.0\times$ healthy / $1.5\times$ minimum thresholds.
- Part 11 lookalike refresh cadence (platform algorithms change ~annually).
- Part 12 QBR template (slide deck structure evolves with org maturity).

For each part, the compliance gate (TS-10 Part 2A contract) holds: **no protected-class features in scoring, routing, or audience definition**. The fair-lensing audit (Part 12 row 9) is the verification of this contract every quarter.